

SOLVING PROBLEMS BY SEARCHING

UNINFORMED SEARCH STRATEGIES

פרק 3

חלק II

מה נלמד היום?

1. Breadth-first search
2. Bidirectional Search
3. Depth-first search
4. Depth-limited search
5. Iterative deepening search
6. Uniform-cost search



© CanStockPhoto.com - esp27635736

BREADTH- FIRST SEARCH (BFS)

Breadth-First Search (BFS)

- מפתח את העלה הכי קרוב לשורש ראשון.

כלומר, מפתח את העץ בשכבות. רק לאחר שפותחו כל השכנים של השורש נעבור לשכנים של השכנים וחוזר חלילה.

שומר העלים בתור FIFO

- סורק באופן מלא את כל מרחב המצבים, אם לא הגענו עדיין אל המטרה.

Breadth-First Search (BFS)

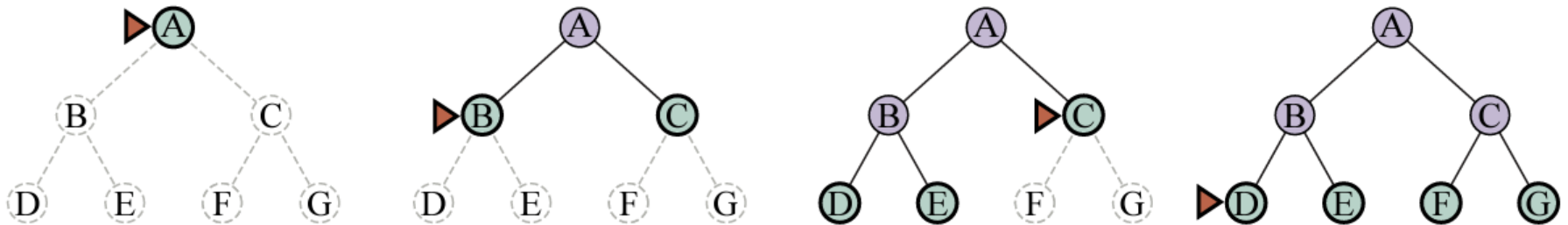


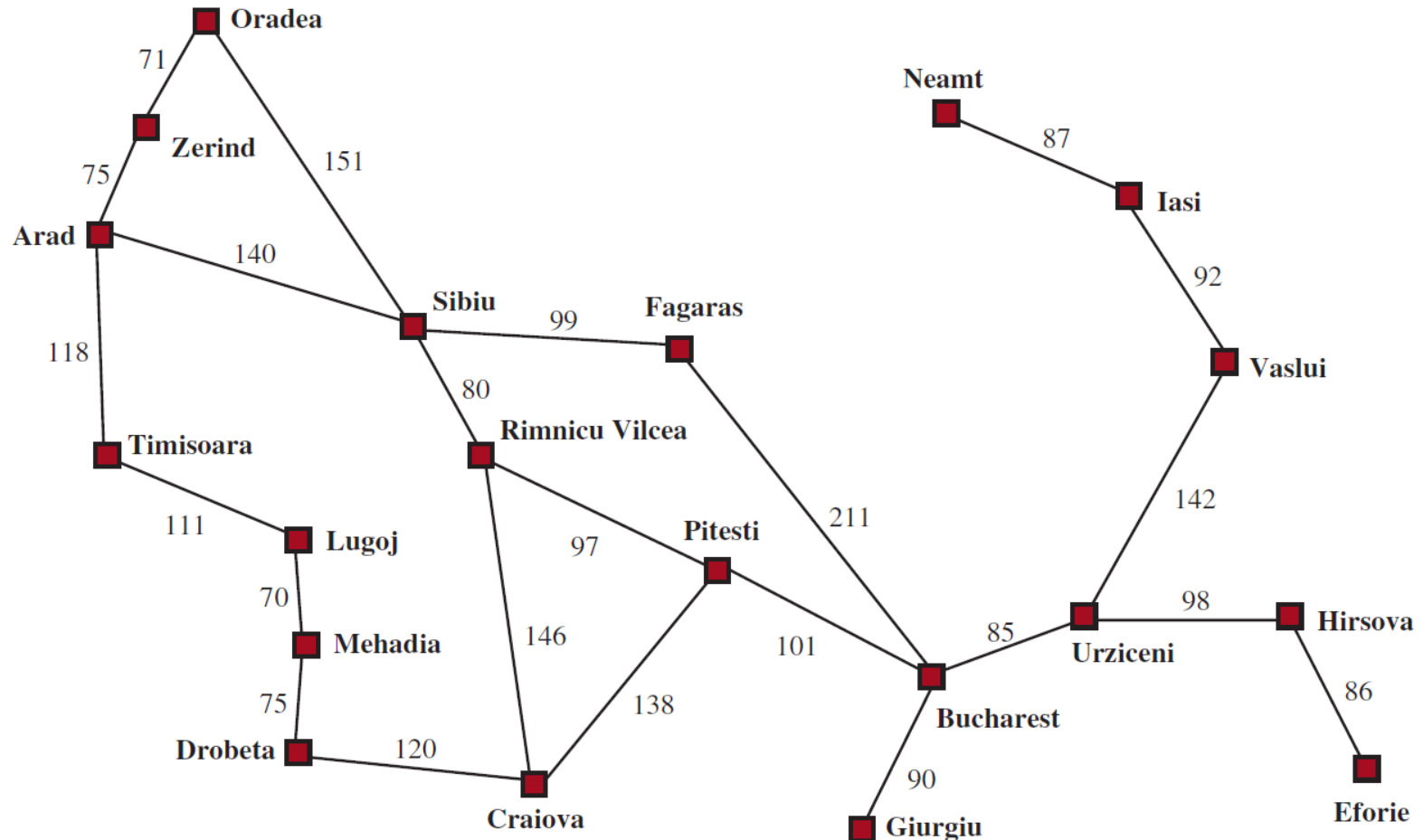
Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

Breadth-First Search (BFS)

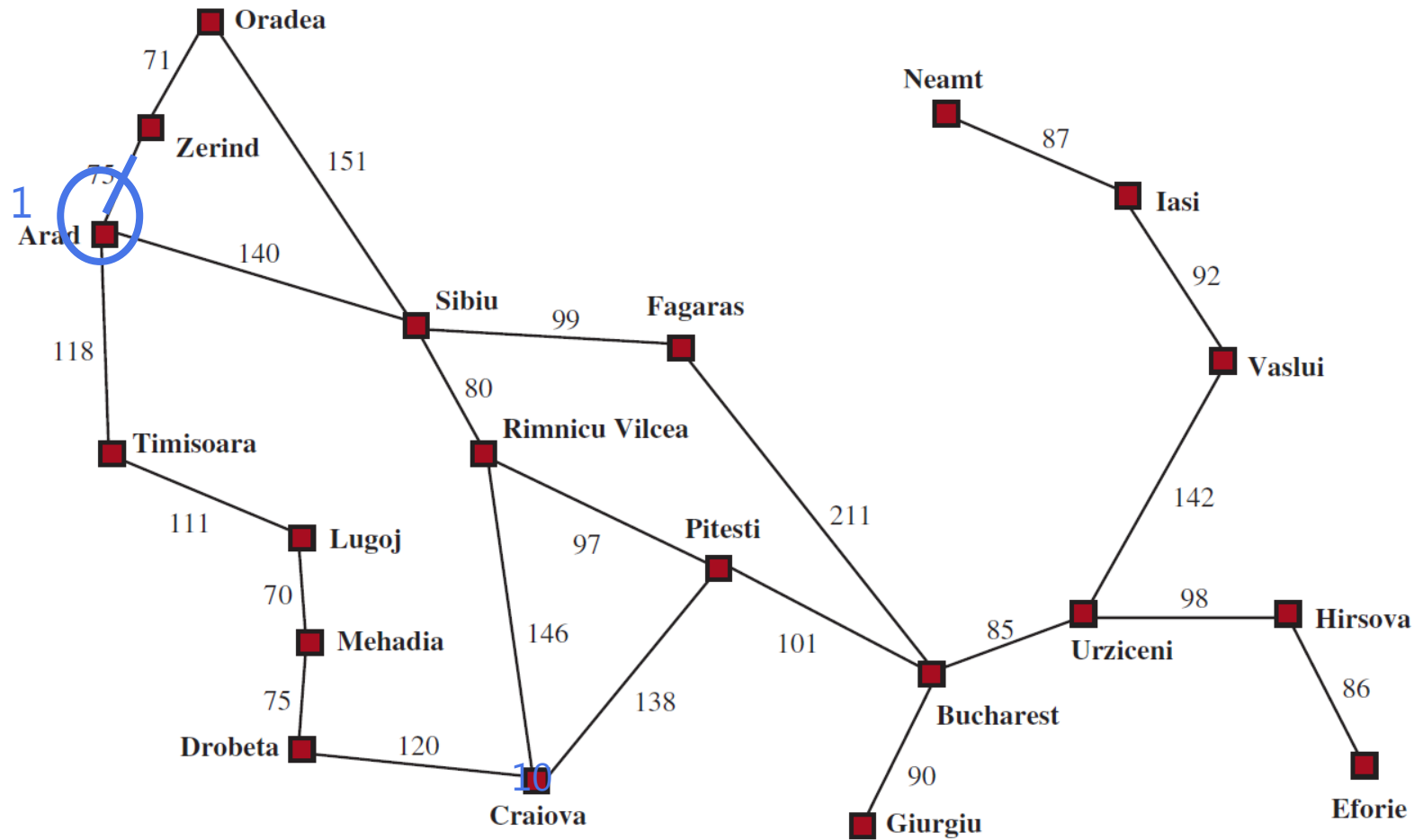
```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  leaves  $\leftarrow$  a FIFO queue, with node as an element  
  
  while not IS-EMPTY( leaves ) do  
    node  $\leftarrow$  Dequeue (leaves)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
  
      add child to leaves  
return failure
```

תרגיל הרצת BFS -

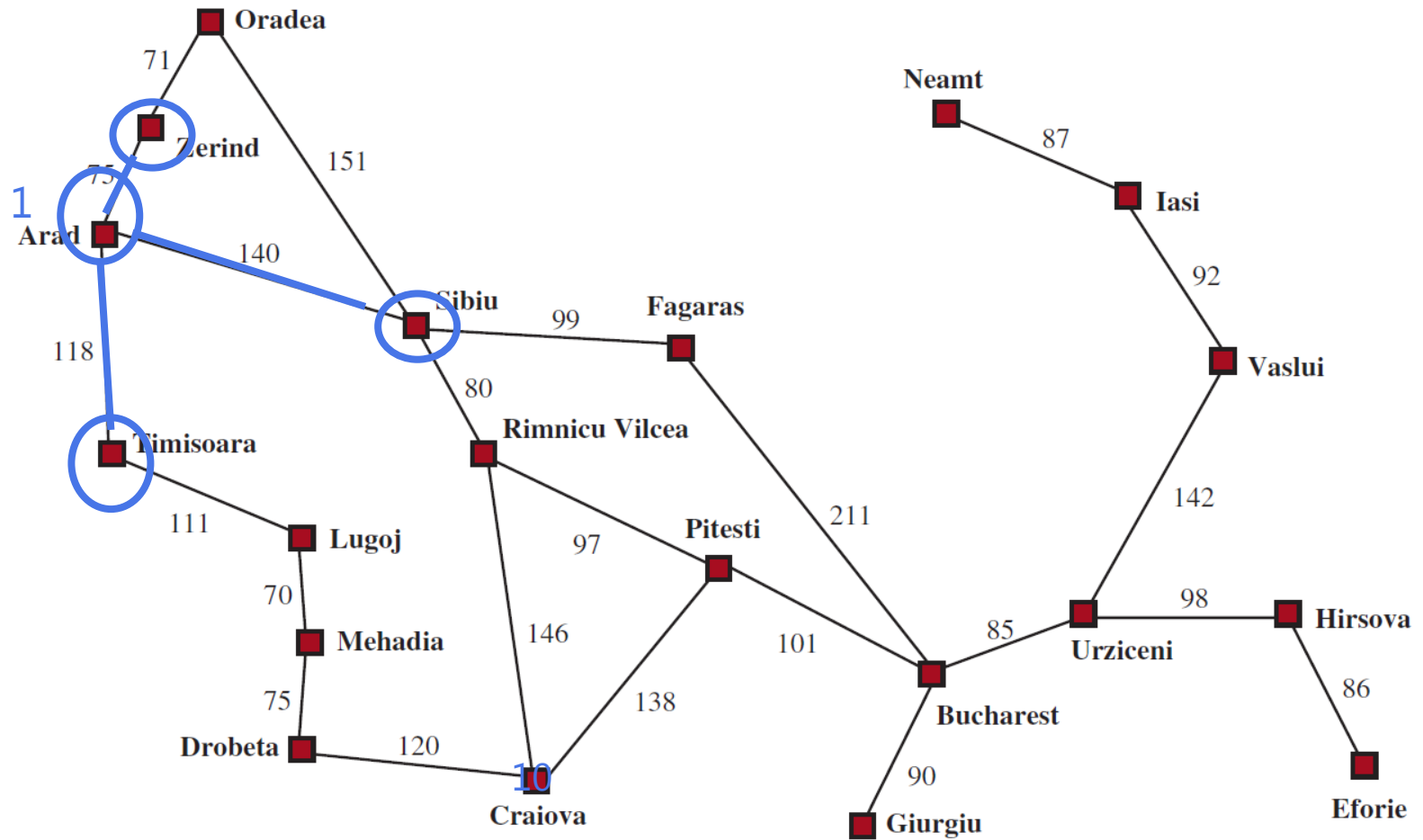
בסדר מילוני- אלפאבתי



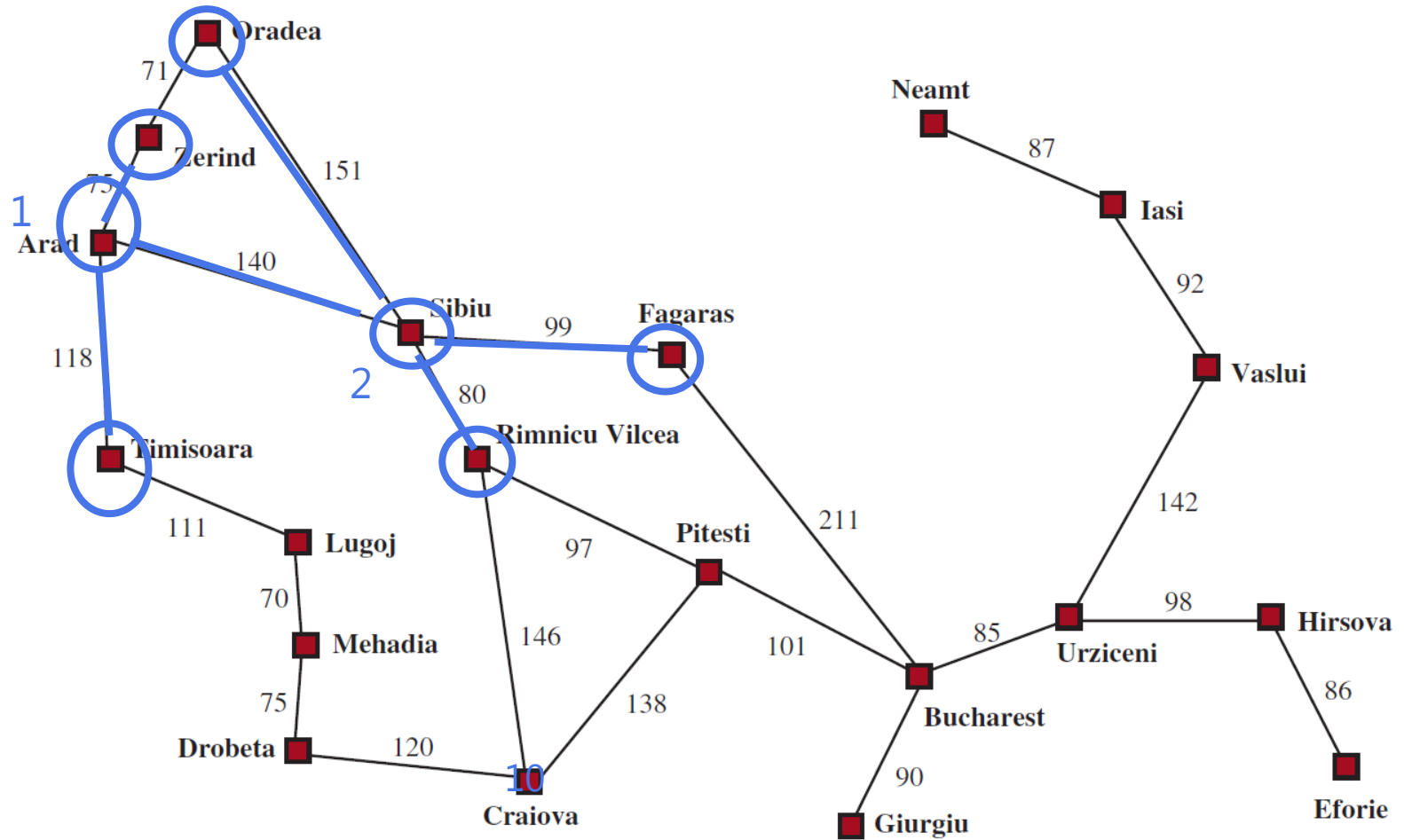
תרגיל הרצת BFS



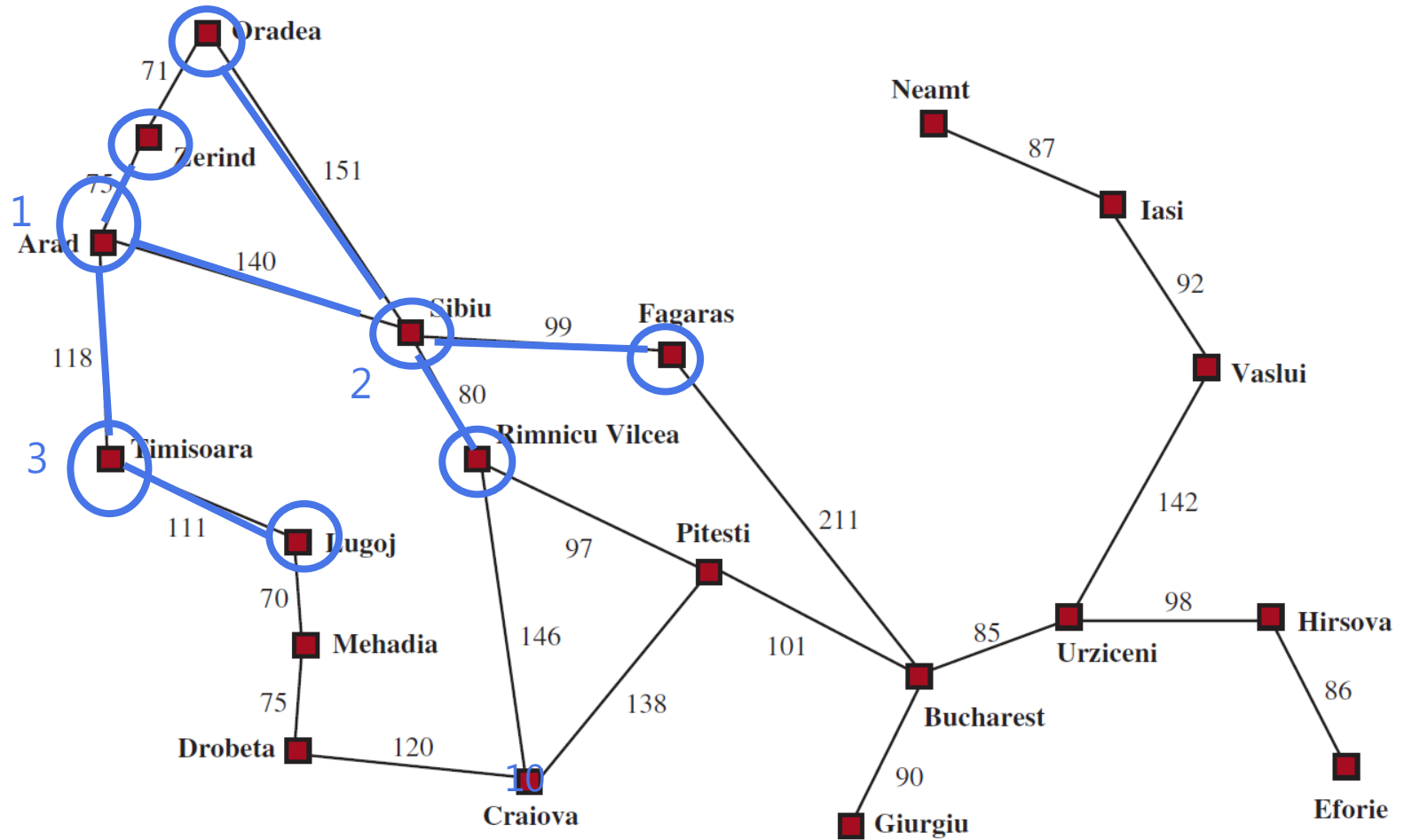
תרגיל הרצת BFS



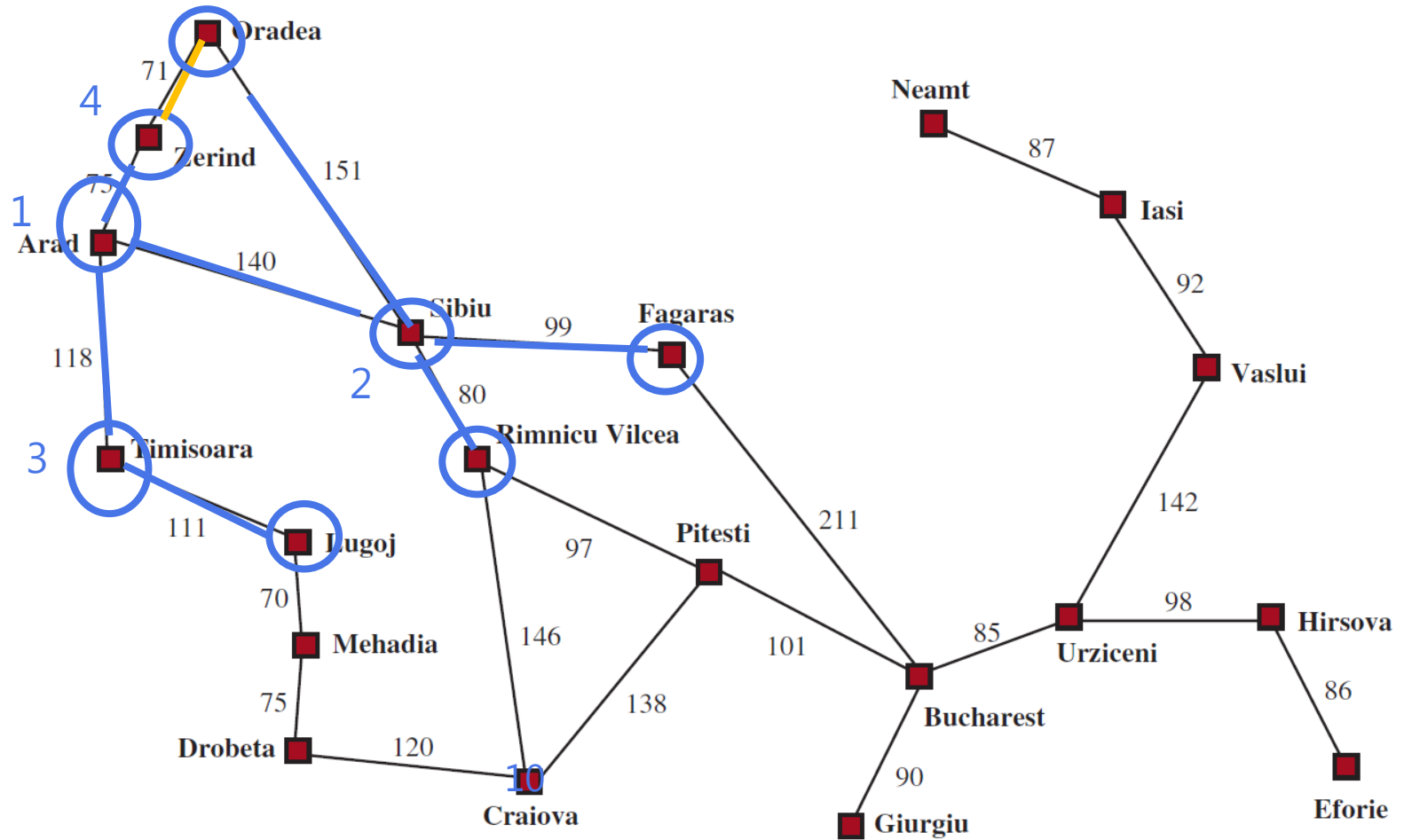
תרגיל הרצת BFS



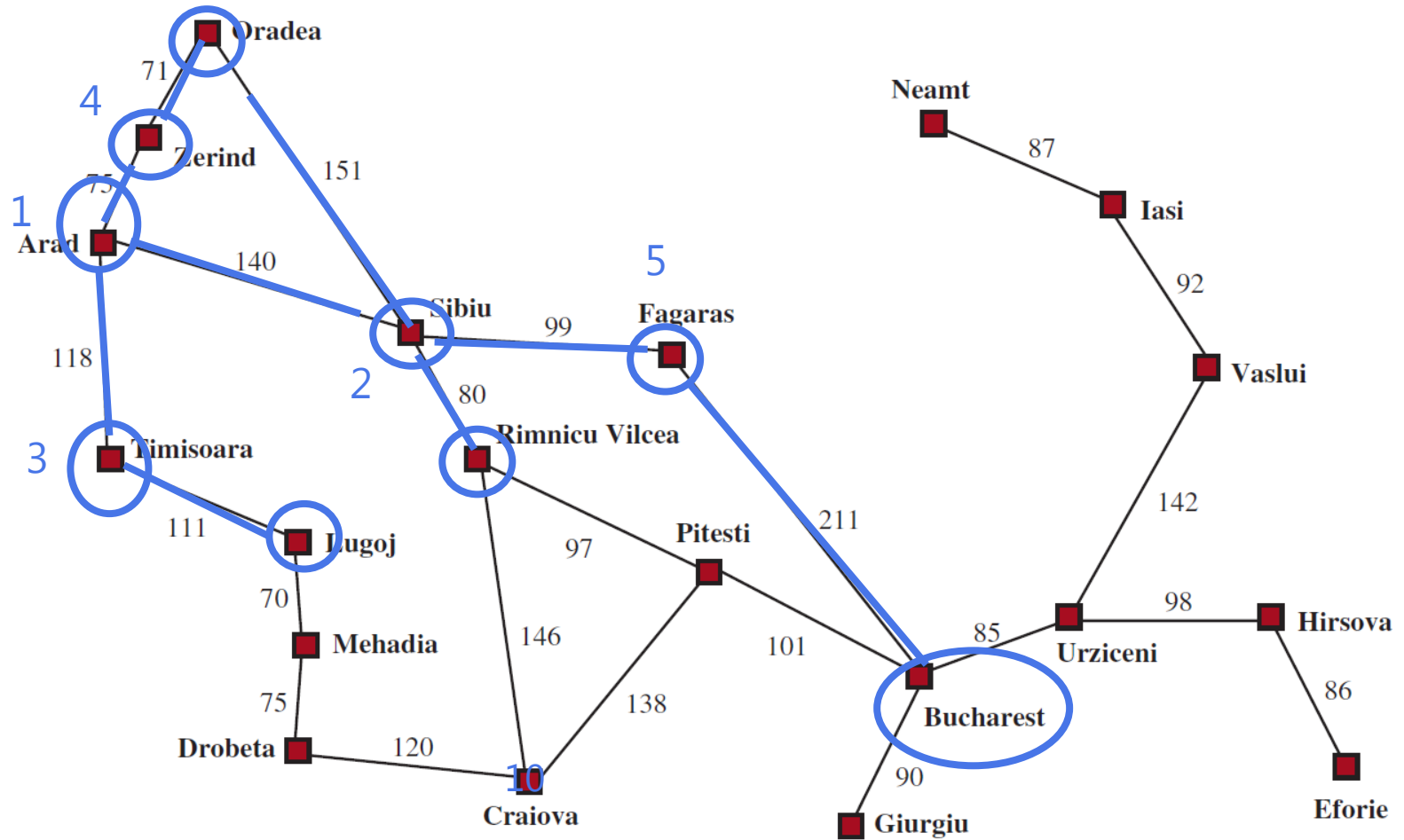
תרגיל הרצת BFS



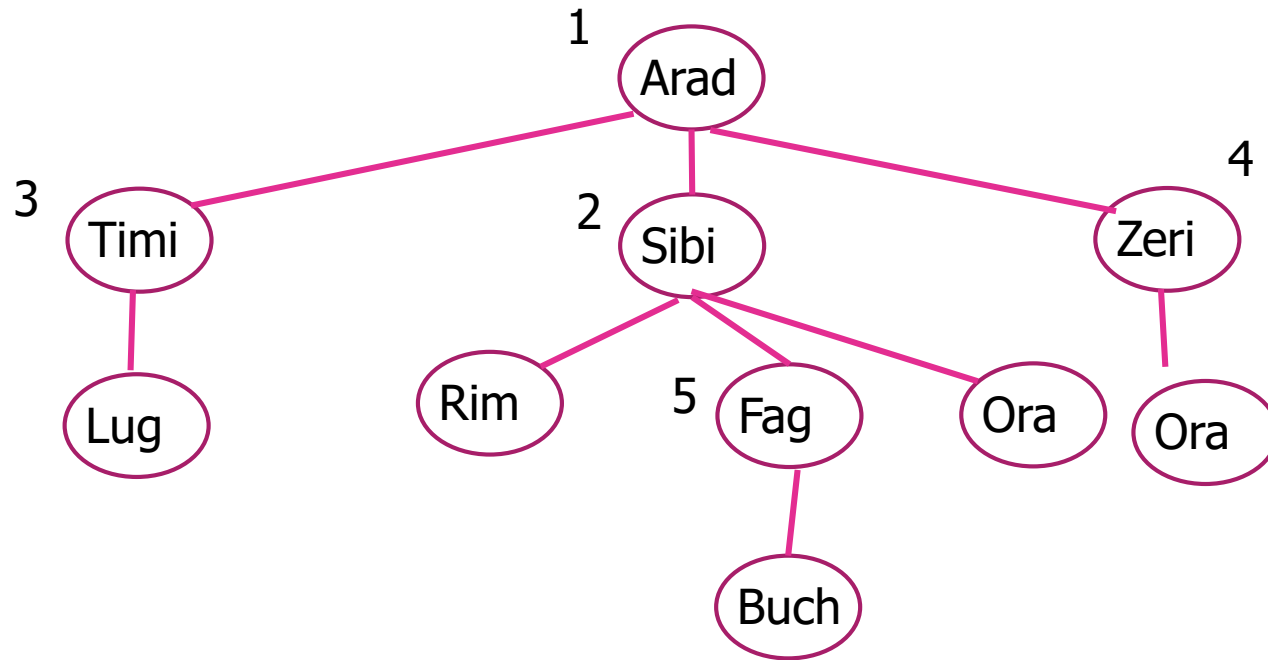
תרגיל הרצת BFS



תרגיל הרצת BFS



סיכום הרצת BFS



Properties of BFS

• **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?

• **אופטימאליות** – האם מוצא פתרון אופטימאלי ביחס לעלות המסלול?

• **סיבוכיות זמן** – כמה זמן צורך האג'?

• **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון

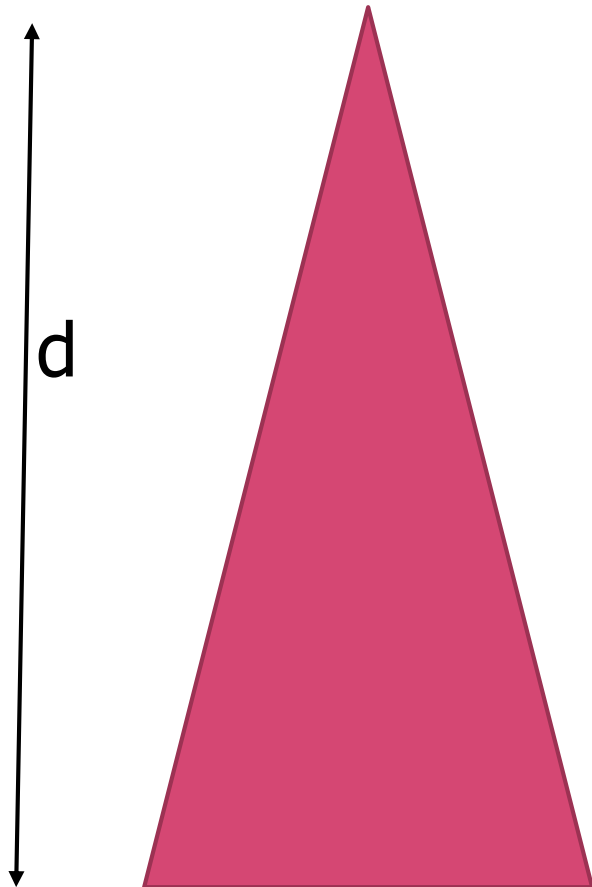
סיבוכיות זמן ומקום נמדדת ביחס למדדים:

b: maximum branching factor of the search tree

d: depth of the least-cost solution

m: maximum depth of the state space (may be ∞)

BFS Time



מספר הקודקודים בעץ

בגובה d

שבו גורם הפיצול הוא b

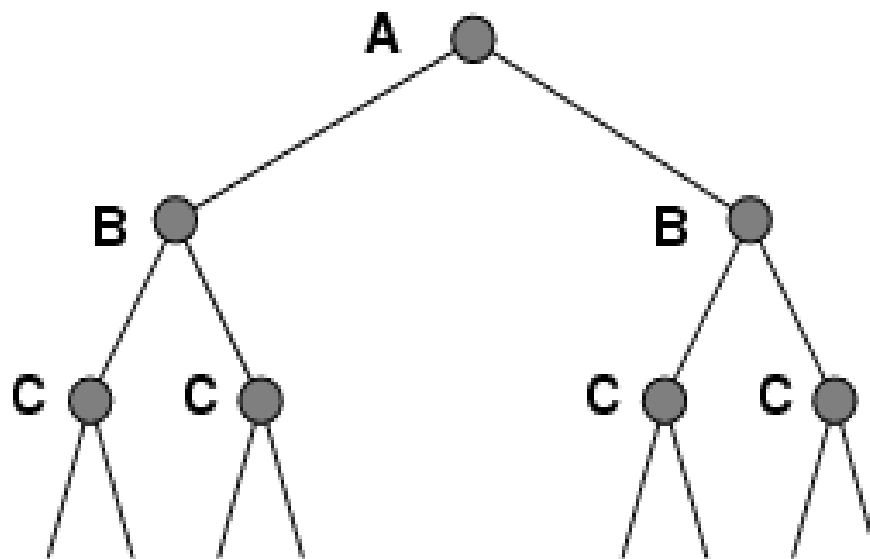
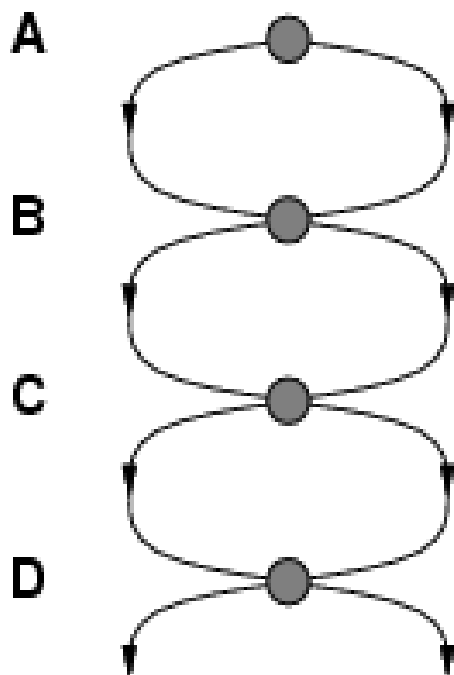
הוא b^d

BFS, $b = 10, 10^4$ nodes/second

depth	nodes	time	memory
2	1100	0.11 sec	1 MB
4	111100	11 sec	106 MB
6	10^7	19 min	10 G
8	10^9	31 hours	1 terabytes
10	10^{11}	129 days	101 terabytes
12	10^{13}	35 years	10 petabytes
14	10^{15}	3523 years	1 exabytes

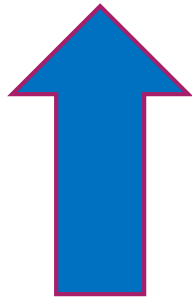
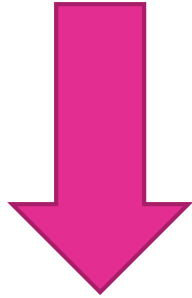
Repeated States

חוסר היכולת לזהות מצבים חוזרים, יכול להפוך בעיה לינארית לבעיה אקספוננציאלית.



Graph Breadth-First Search (BFS)

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  leaves  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY( leaves ) do  
    node  $\leftarrow$  POP( leaves )  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to leaves  
  return failure
```



BIDIRECTIONAL SEARCH

Bidirectional Search

• מנהלים חיפוש קדימה מהמצב ההתחלתי
ובמקביל חיפוש אחורה מהמצב הסופי ועוצרים
כששניהם נפגשים.

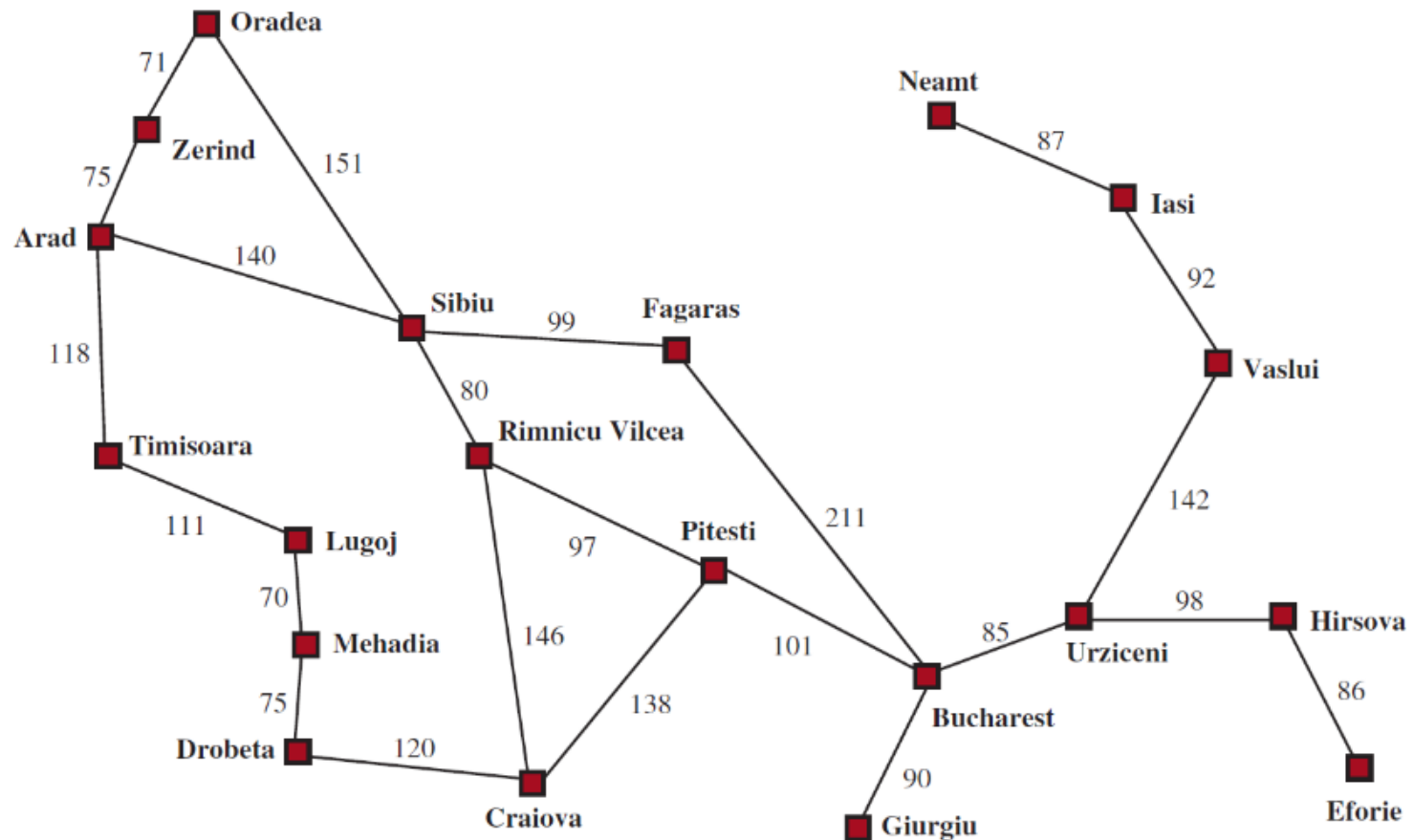
• בכל שלב בודקים האם הקוד' הבא נמצא בצד
השני – אם כן יש לנו מסלול

Bidirectional Search

לכל צד בחיפוש (קדימה / אחורה)

- יש תור של עלים
- יש טבלת גיבוב של קודקודים סגורים.
- כשמייצרים קודקוד בודקים האם הוא נמצא בטבלת הסגורים או בעלים של הצד השני אם כן משלבים המסלולים.
- אם לא, מוסיפים אותו לתור העלים של הצד שלו.

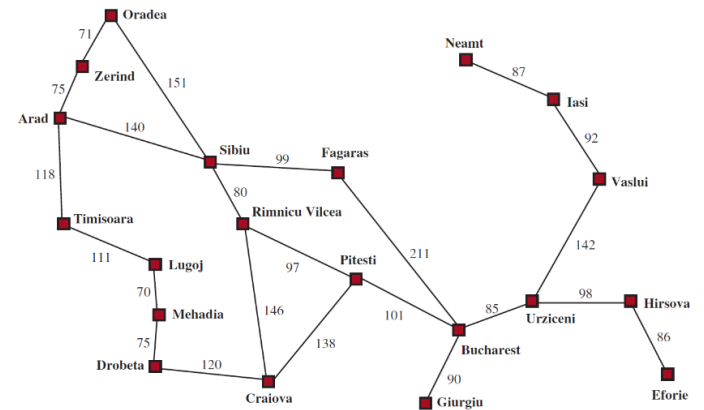
תרגיל הרצת Bidirectional Search



סיכום הרצת Bidirectional

1

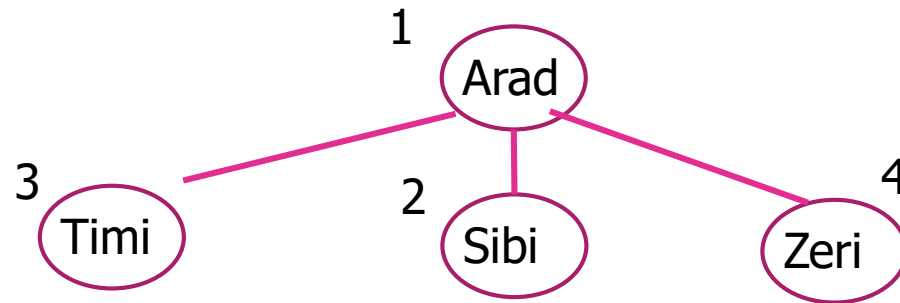
Arad



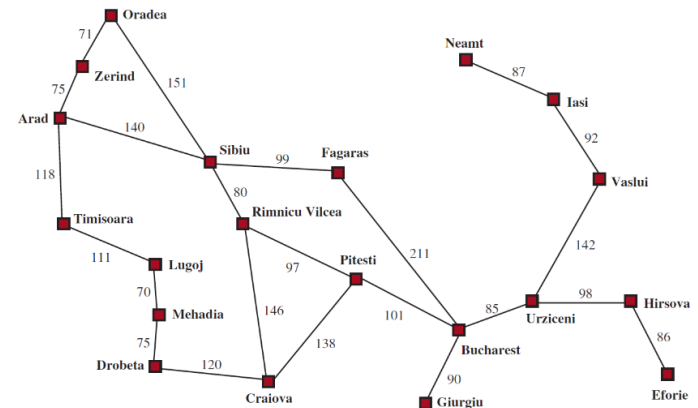
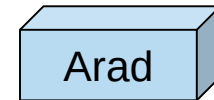
Buch

2

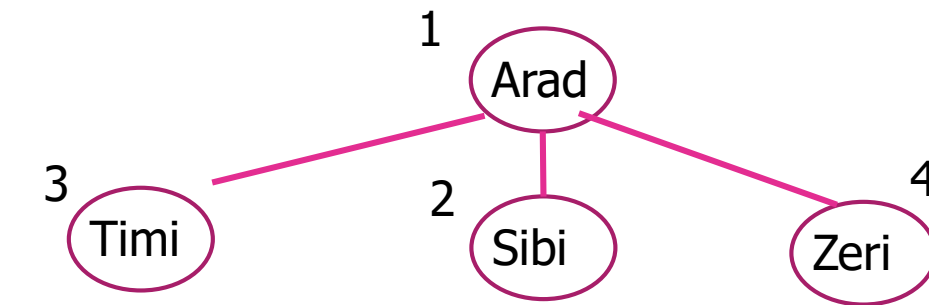
סיכום הרצת Bidirectional



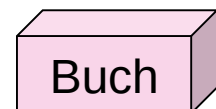
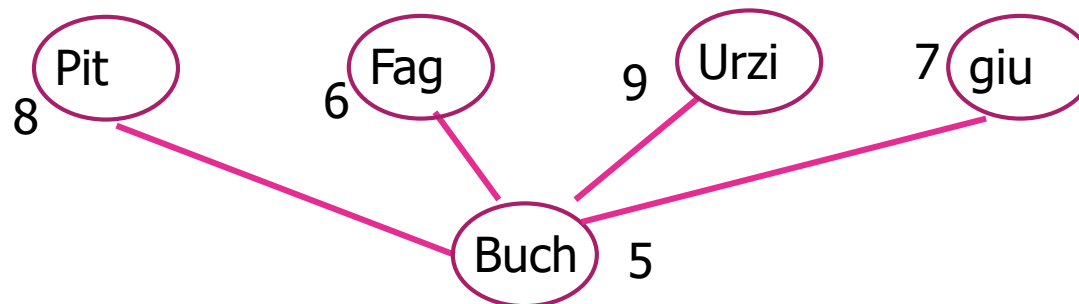
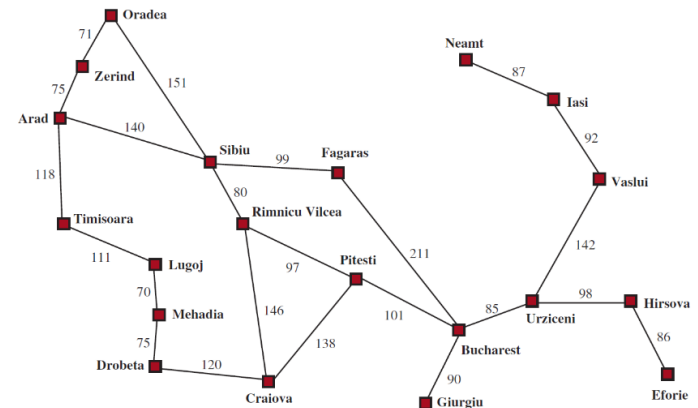
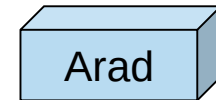
Closed List



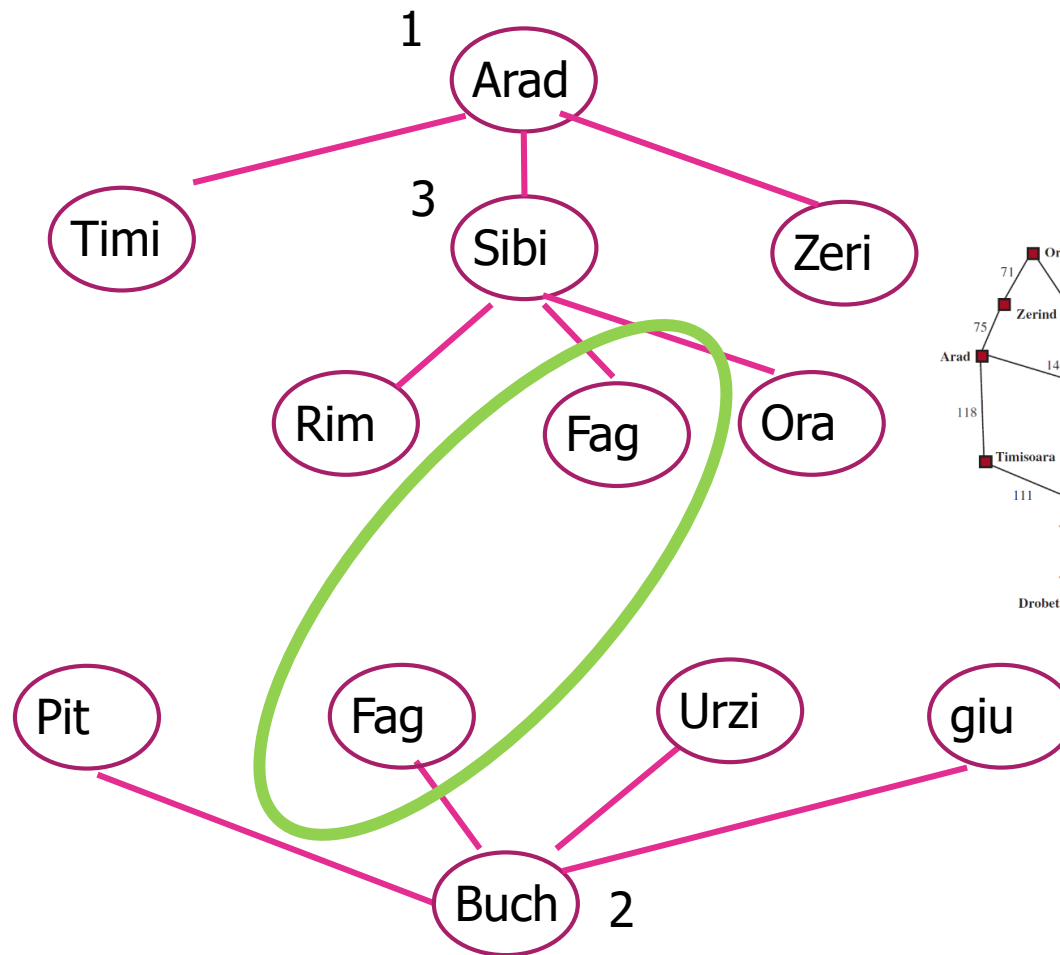
סיכום הרצת Bidirectional



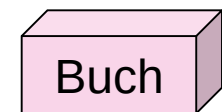
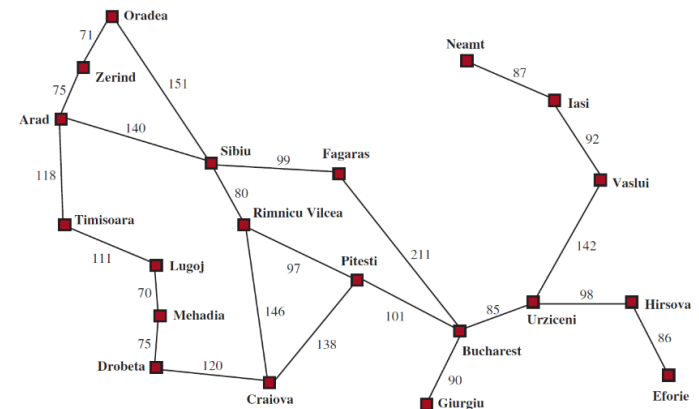
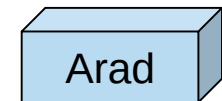
Closed List



סיכום הרצת Bidirectional



Closed List



Bidirectional Search

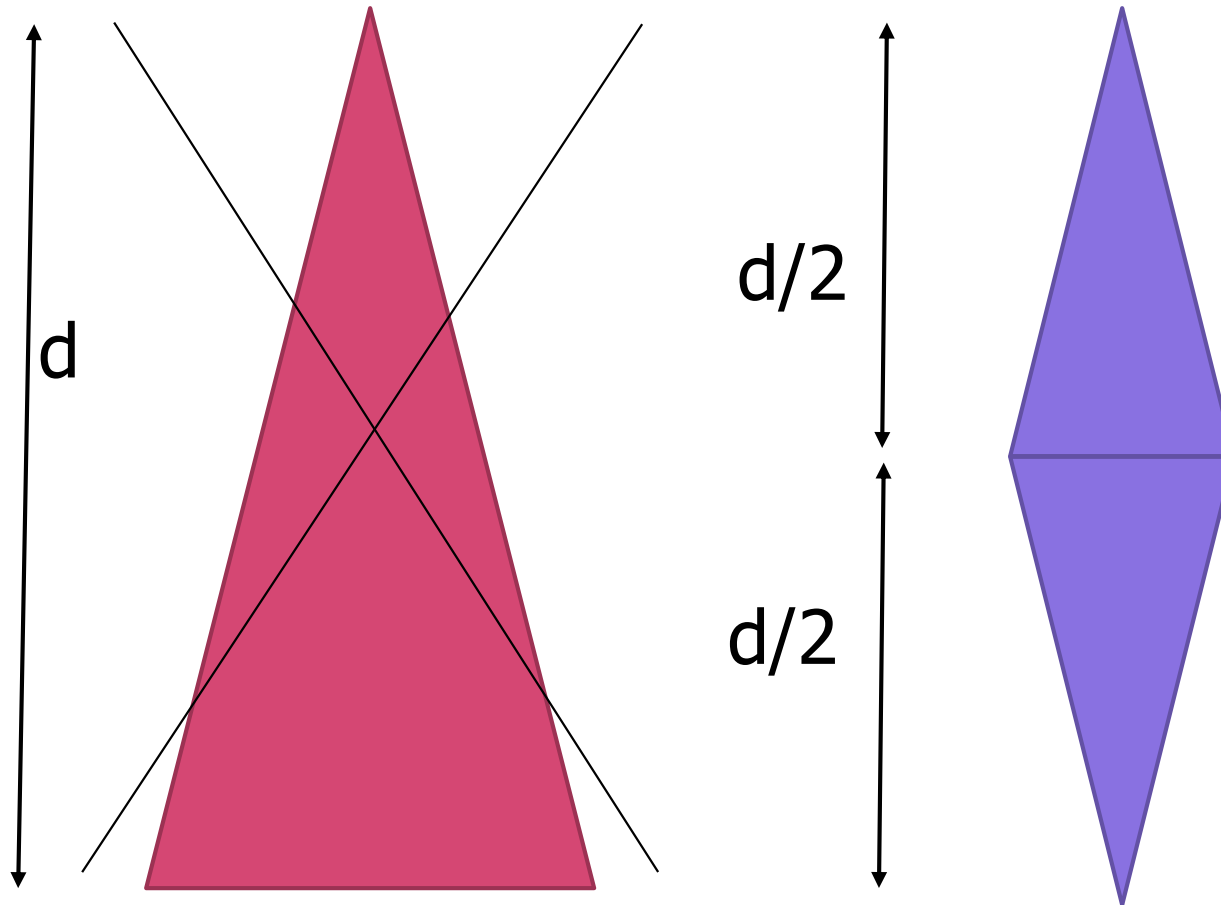
• **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?

• **אופטימאליות** – האם מוצא פתרון אופטימאלי
ביחס לעלות המסלול?

• **סיבוכיות זמן** – כמה זמן צורך האג'?

• **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון

Bidirectional Search Time





DEPTH- FIRST SEARCH (DFS)

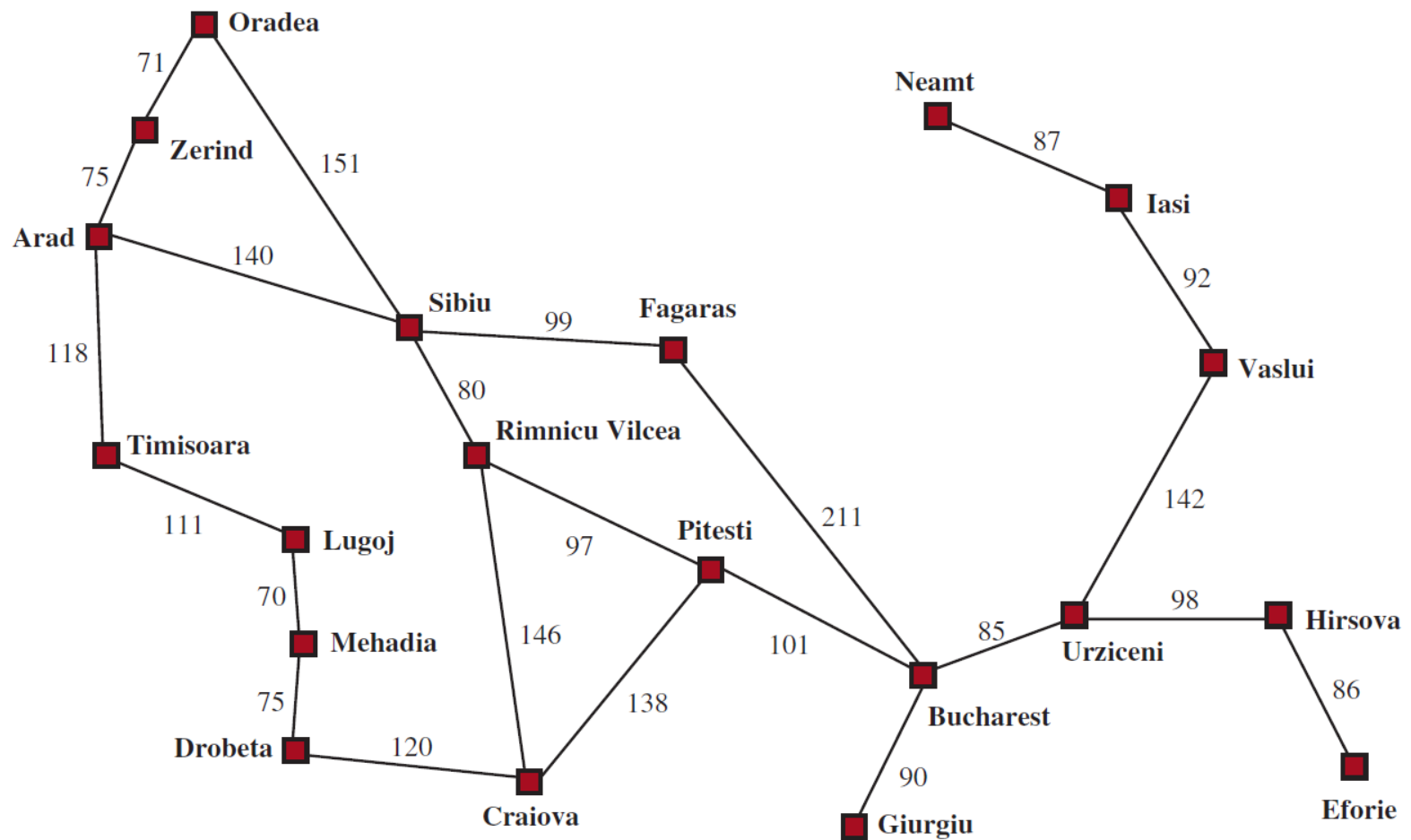
Depth-First Search (DFS)

- מפתח את העלה הכי עמוק ראשון.

שומר העלים במחסנית LIFO

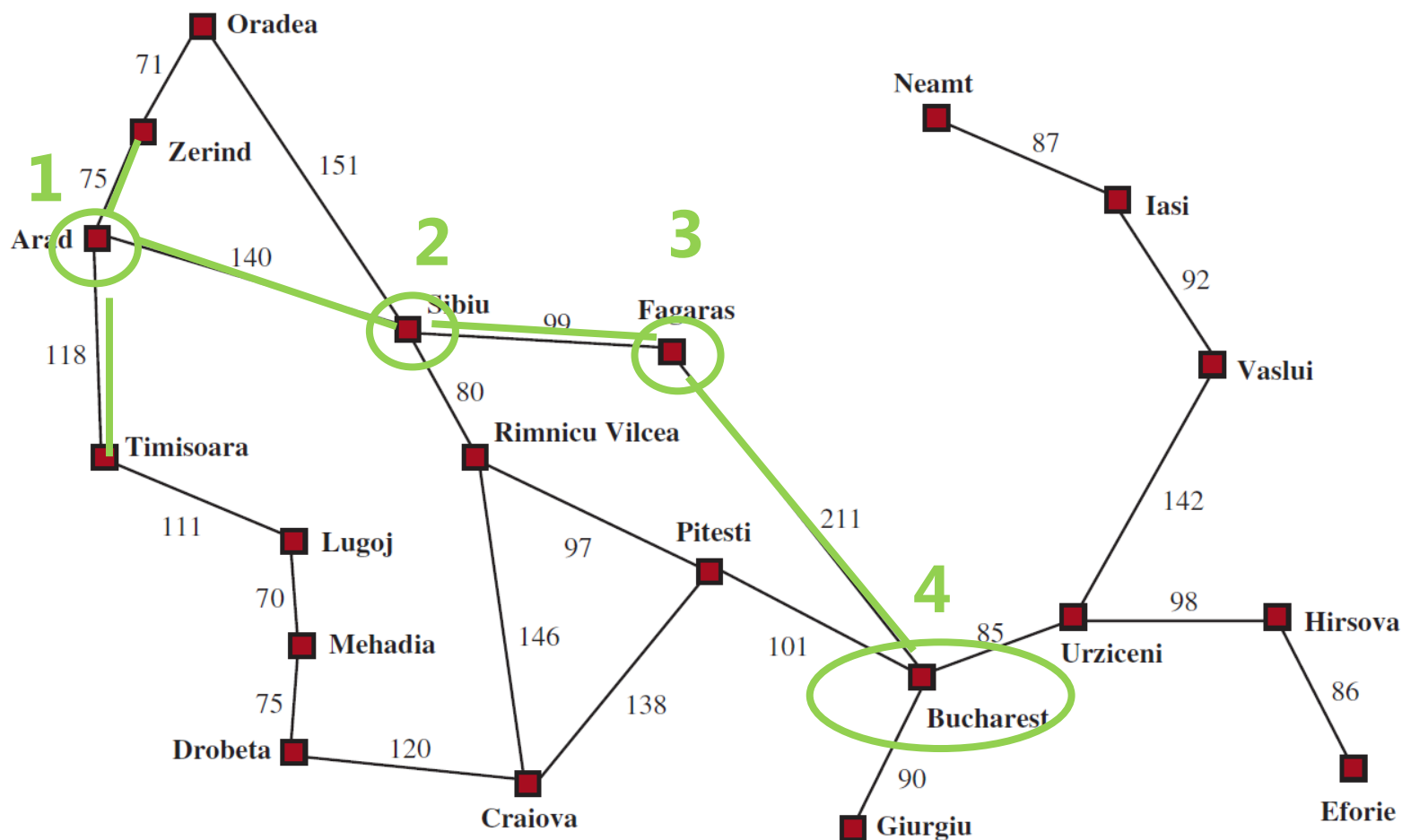
- כשמגיעים לעלה שאין לו המשך, הולכים צעד אחורה עד שמגיעים לקוד' העמוק ביותר הבא בעץ.

תרגיל הרצת DFS

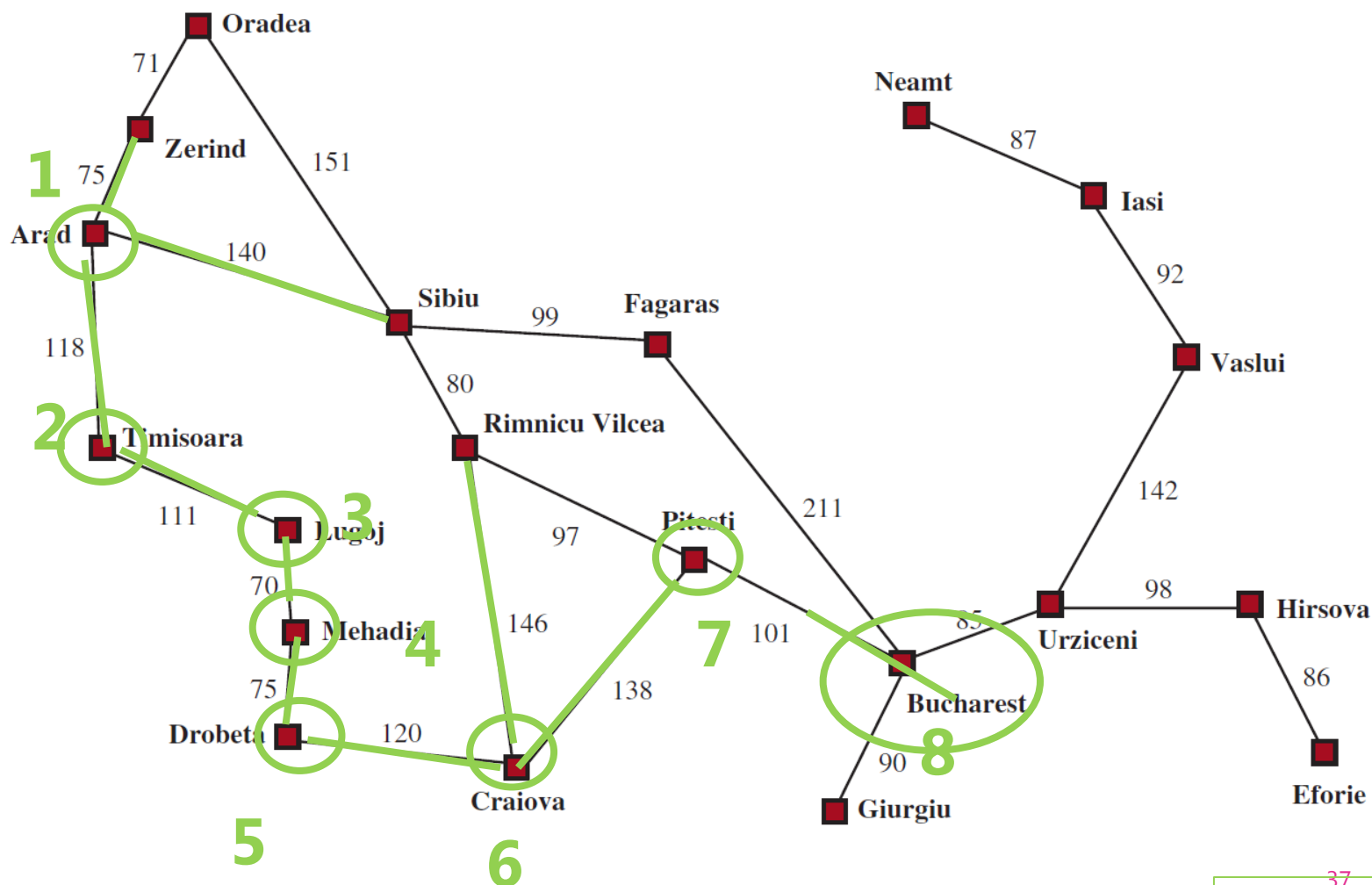


סדר מילוני

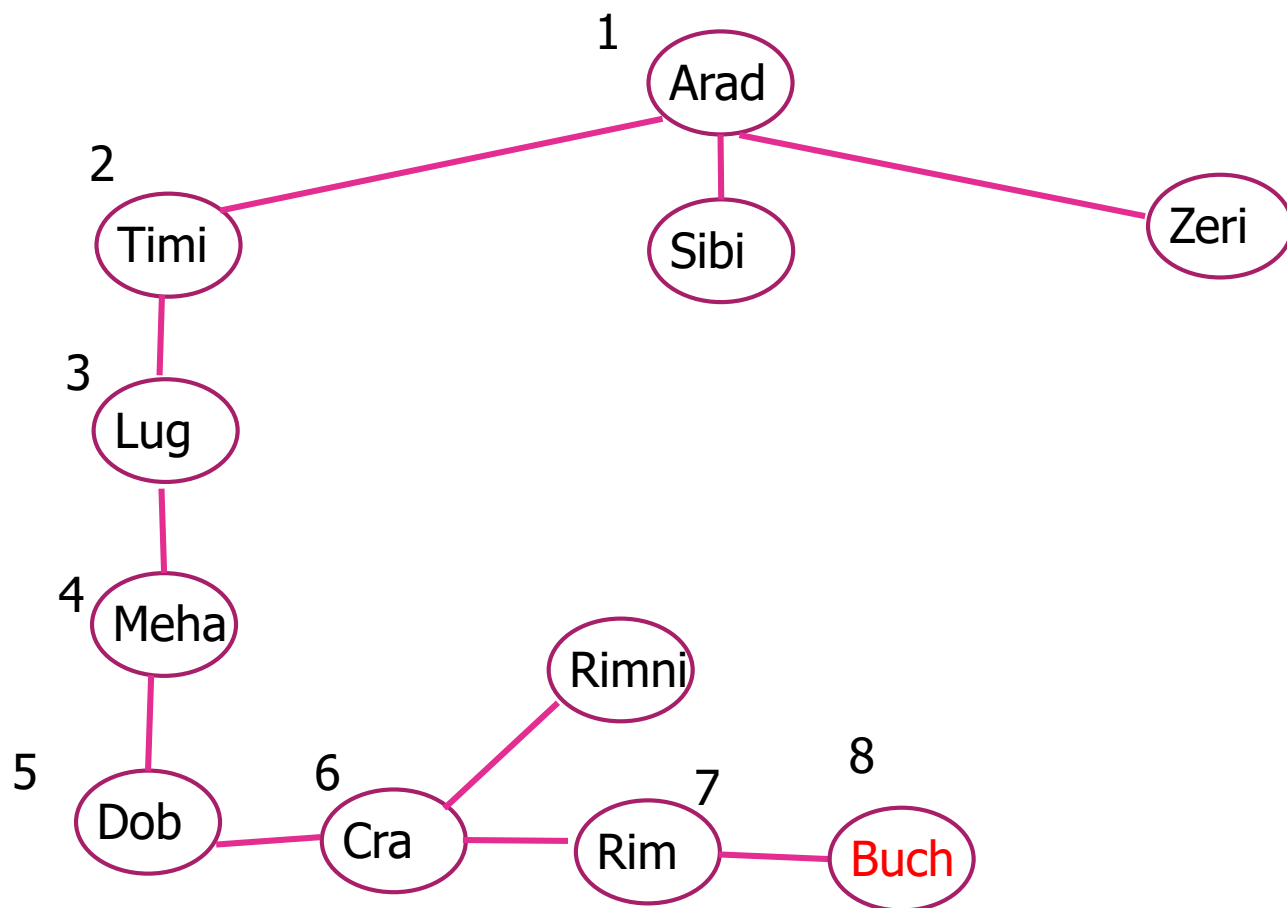
תרגיל הרצת DFS



תרגיל הרצת DFS - לא לפי סדר מילוני



סיכום הרצת DFS – לא לפי סדר מילוני



Depth-First Search Properties

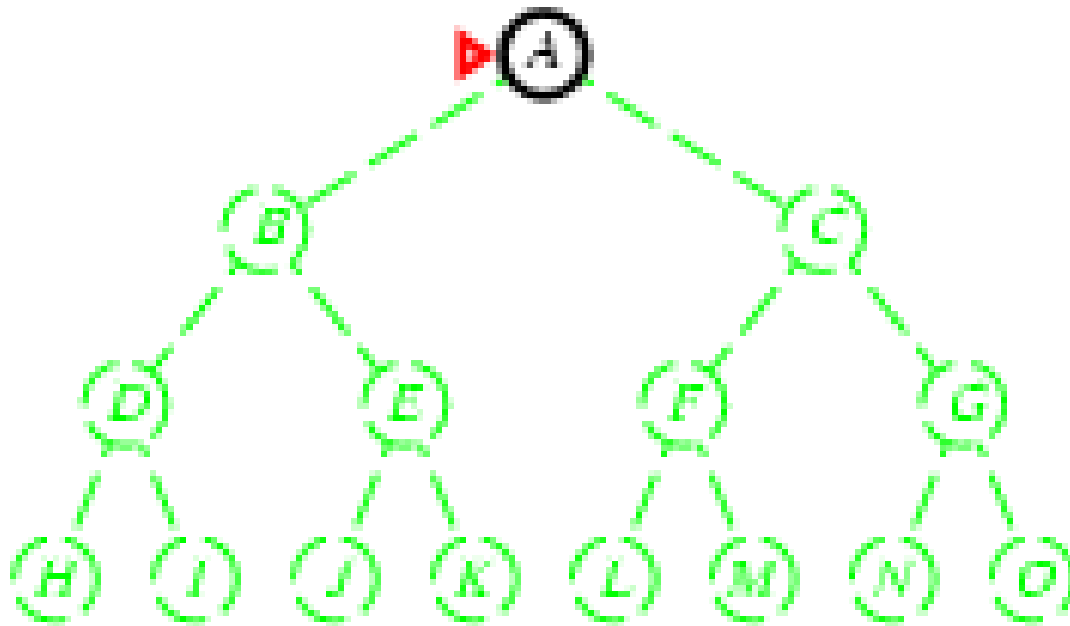
• **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?

• **אופטימאליות** – האם מוצא פתרון אופטימאלי ביחס לעלות המסלול?

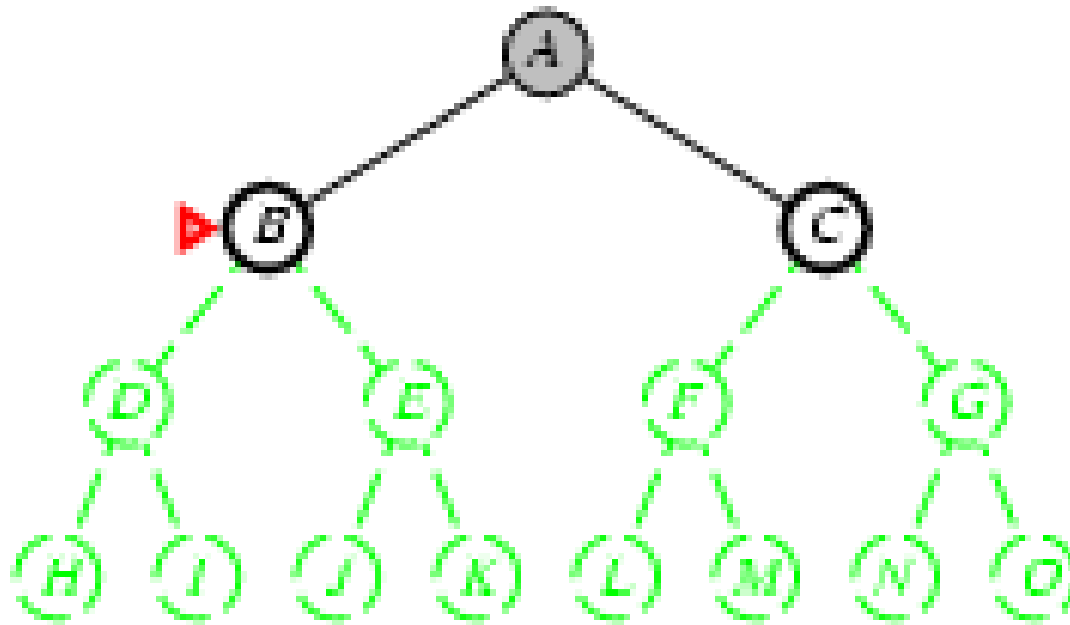
• **סיבוכיות זמן** – כמה זמן צורך האג'?

• **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון

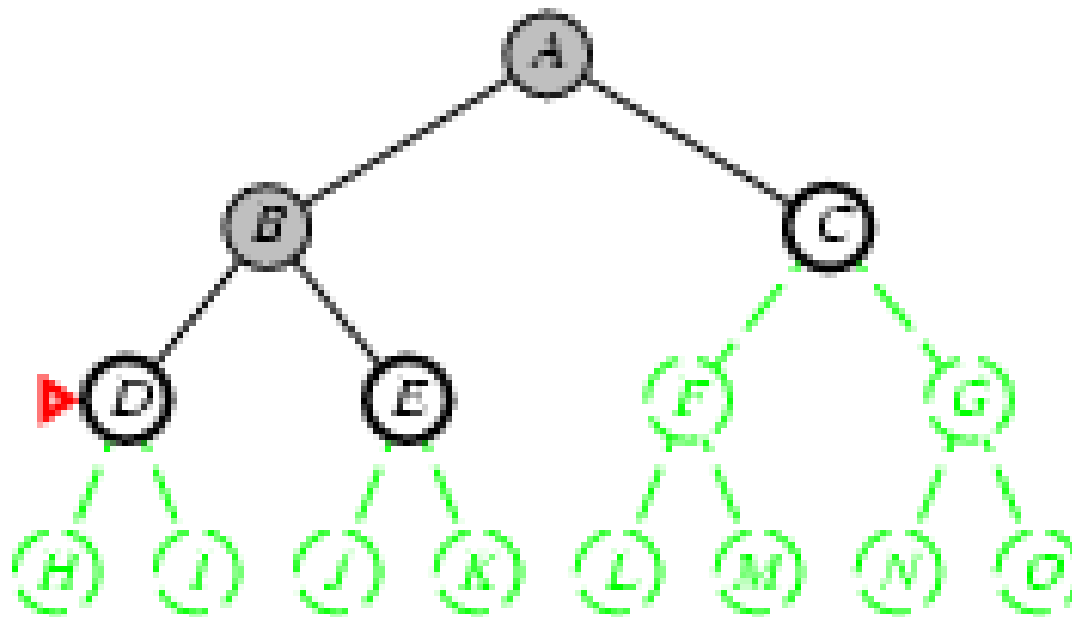
Depth-First Search (DFS)



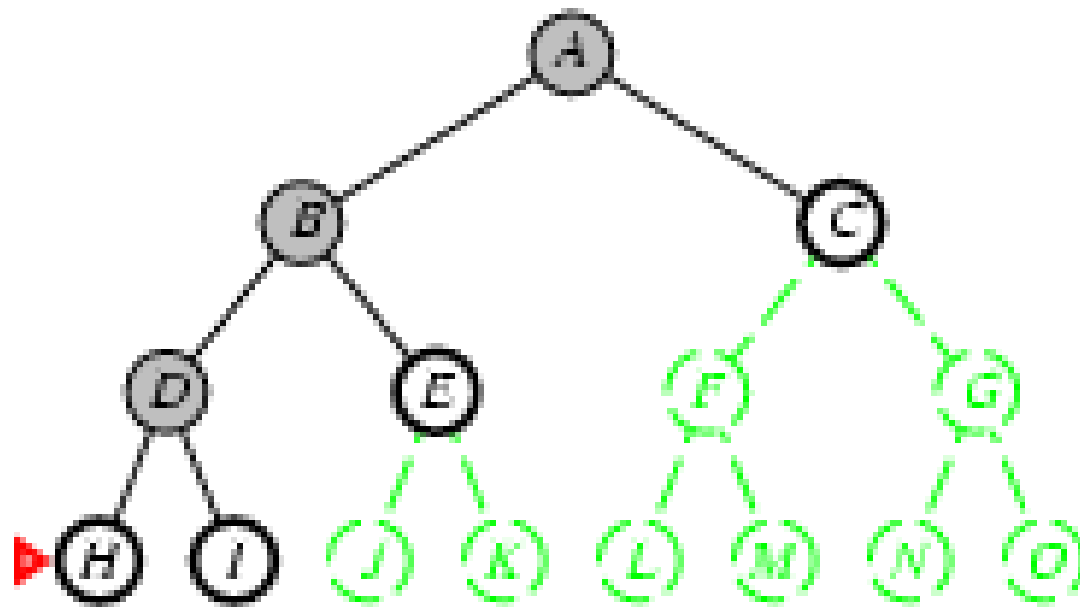
Depth-First Search (DFS)



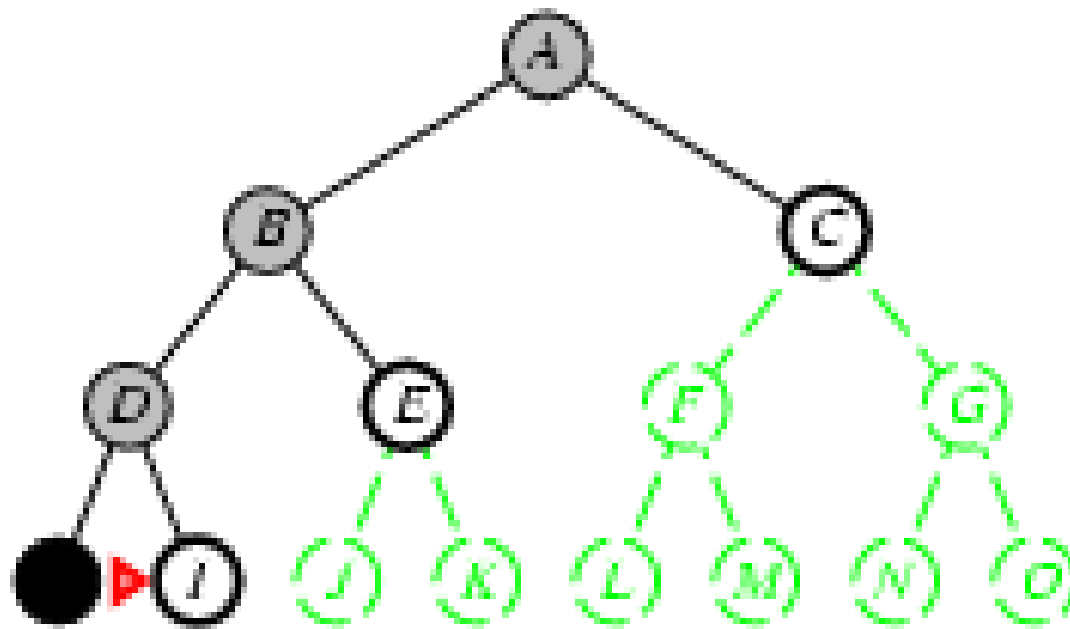
Depth-First Search (DFS)



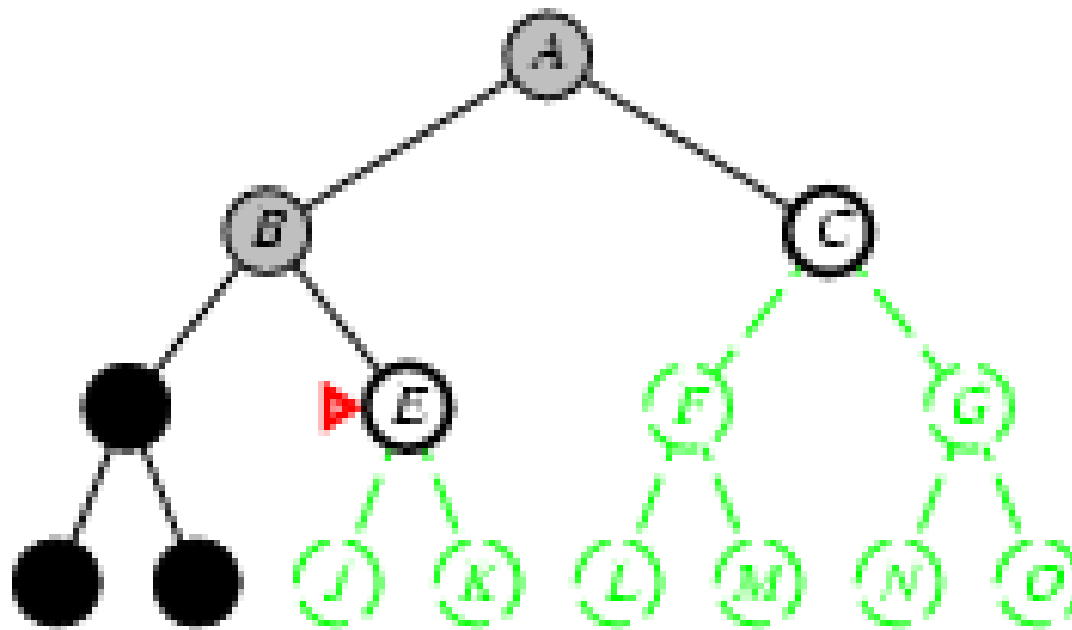
Depth-First Search (DFS)



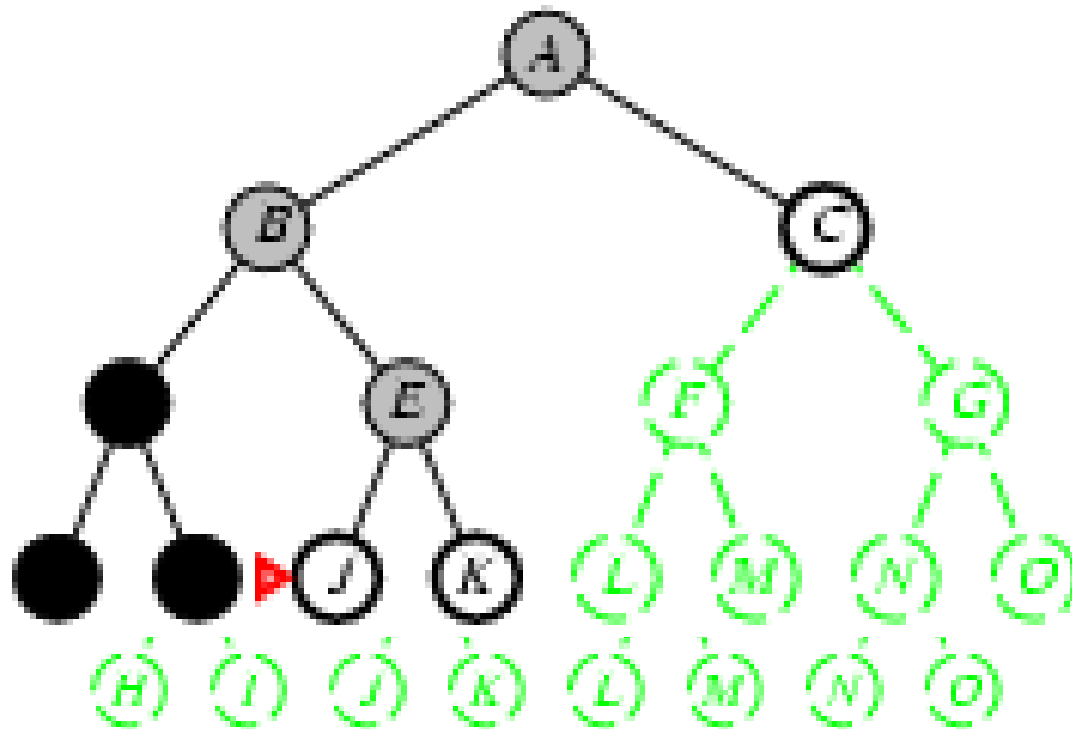
Depth-First Search (DFS)



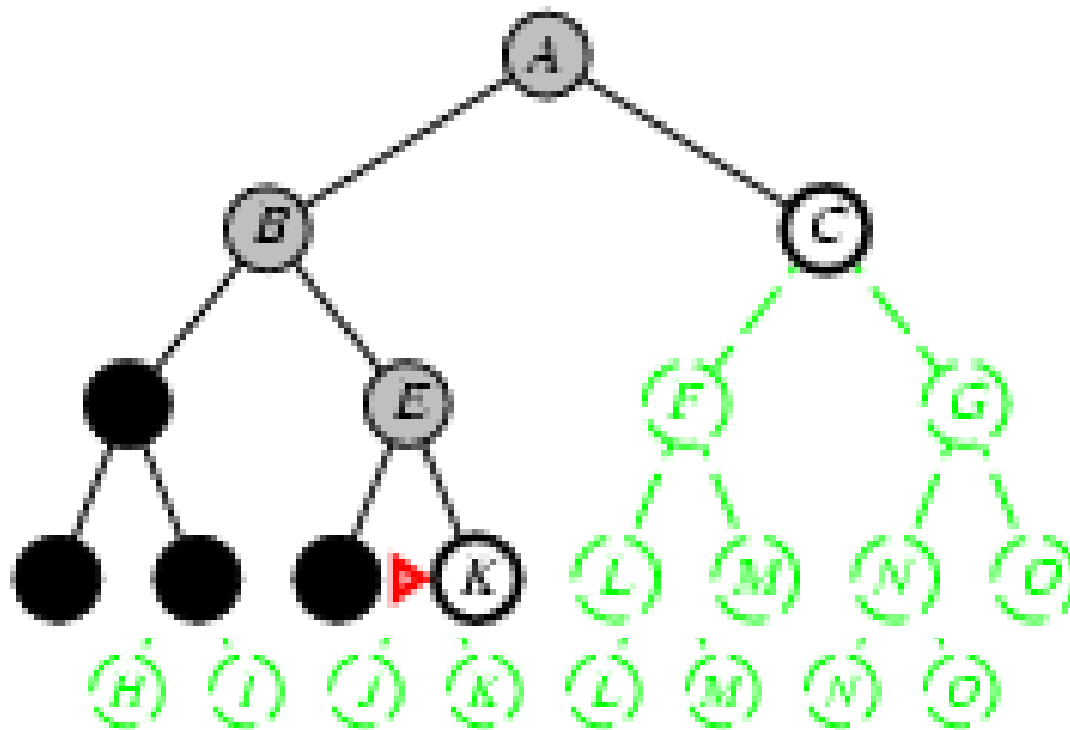
Depth-First Search (DFS)



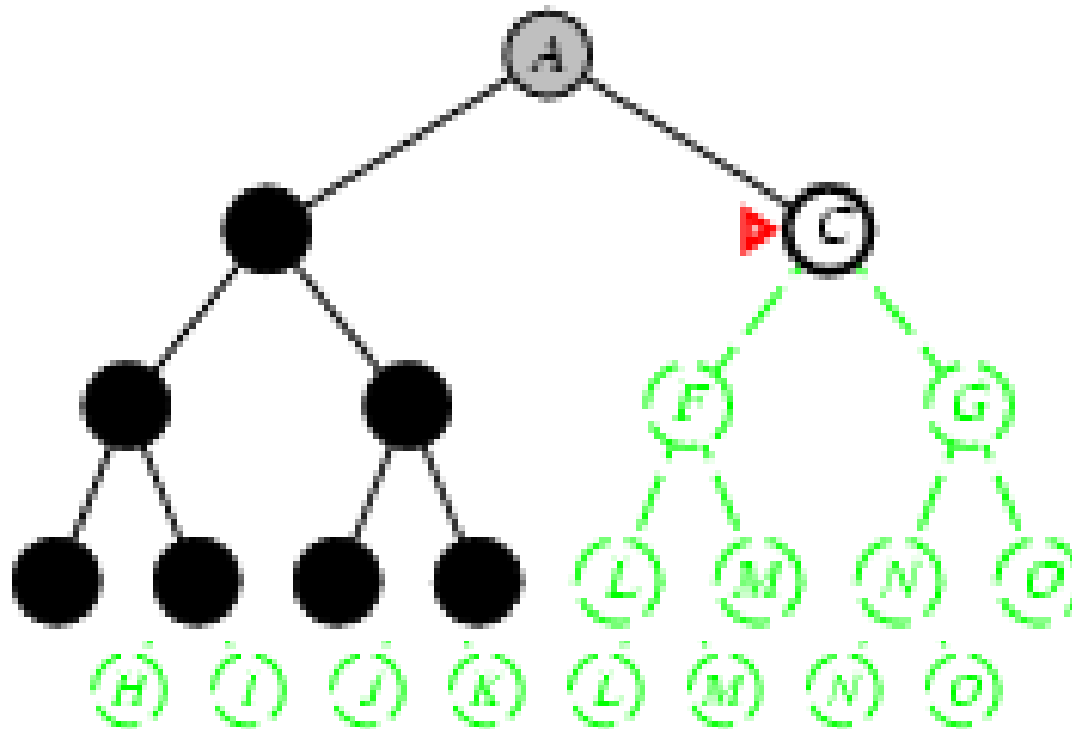
Depth-First Search (DFS)



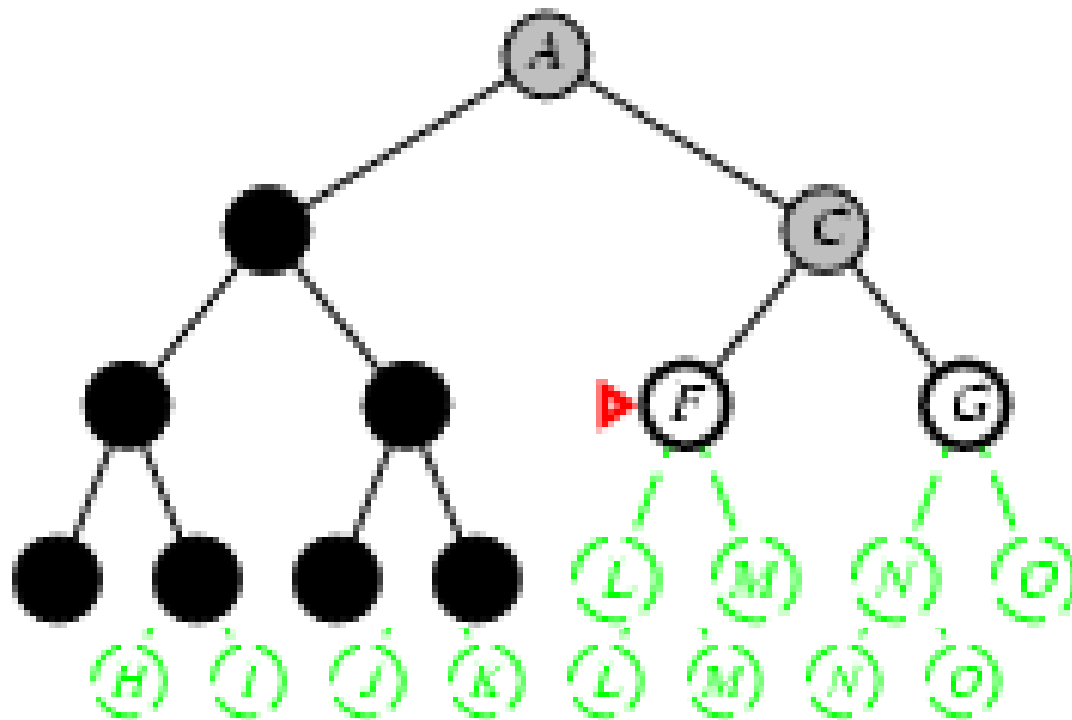
Depth-First Search (DFS)



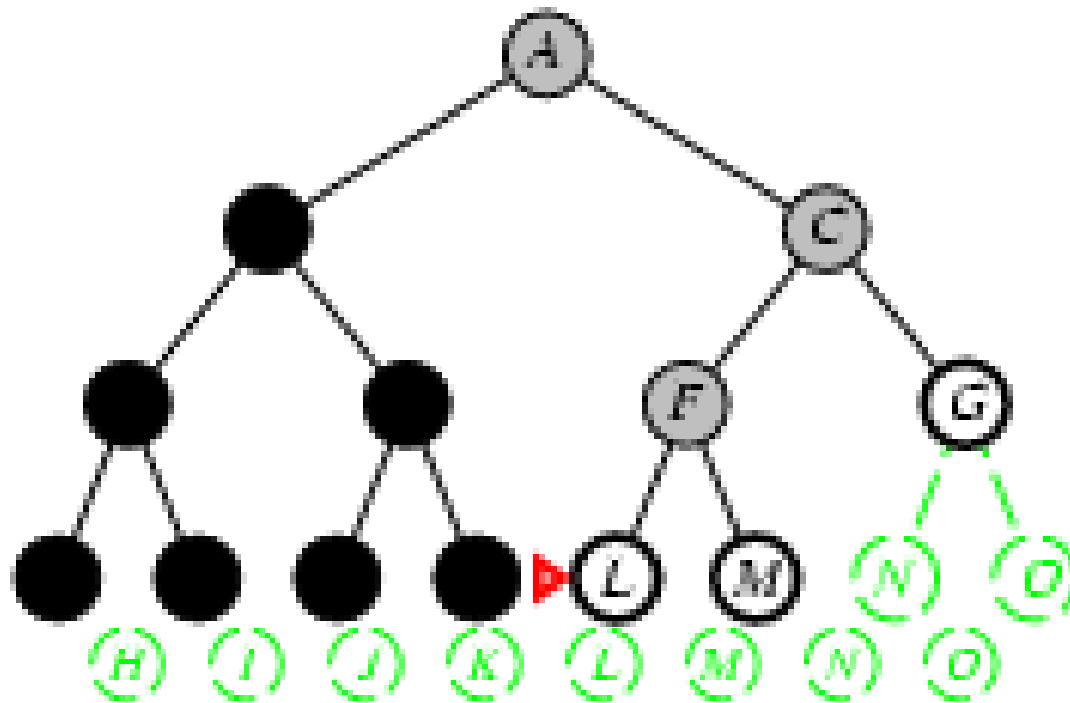
Depth-First Search (DFS)



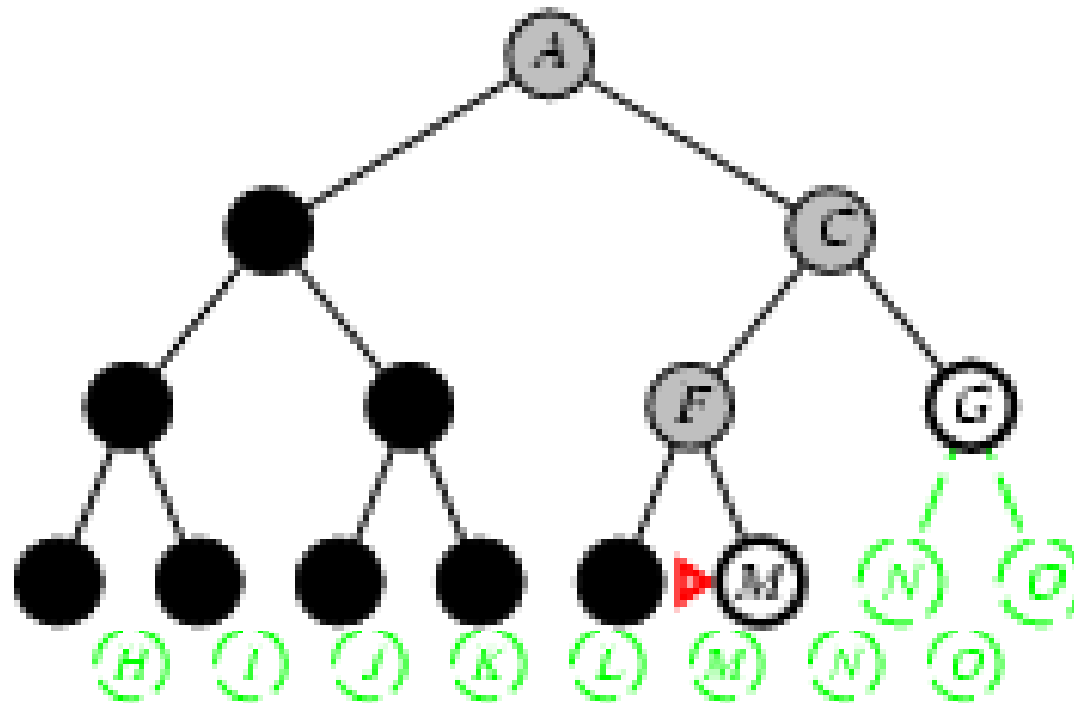
Depth-First Search (DFS)



Depth-First Search (DFS)



Depth-First Search (DFS)



. למחשבה

כיצד נשלב יתרונות

DFS ו BFS?





DEPTH- LIMITED SEARCH

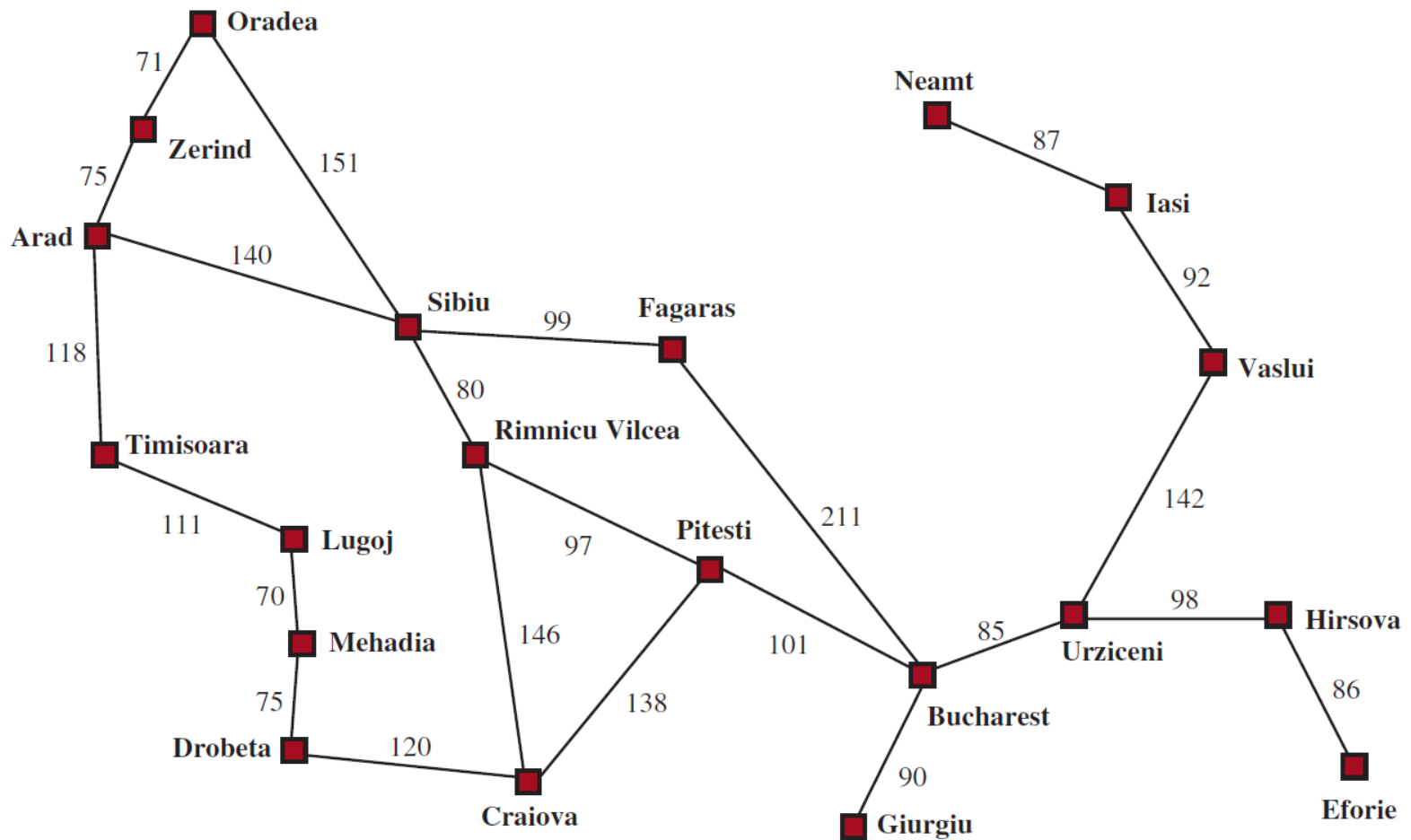
Depth-Limited Search

- כדי למנוע אינסופיות העץ של DFS, מוסיפים **פרמטר גובה מלאכותי** של העץ. כלומר לקודקוד ברמה l מבחינתנו אין ילדים.

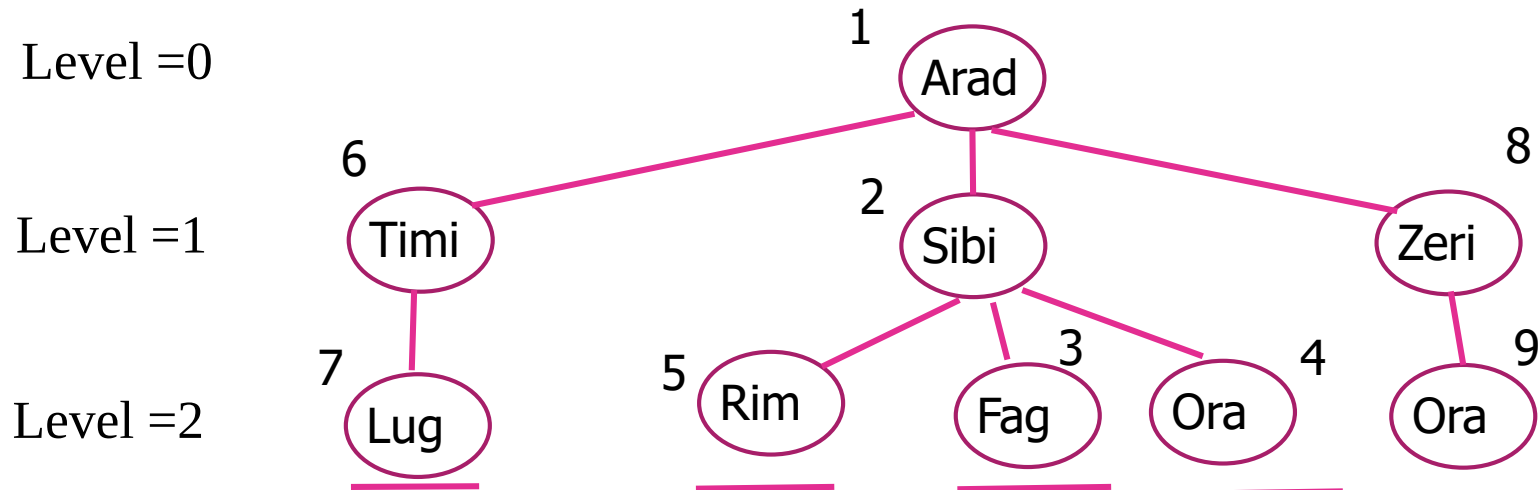
DepthLimitedSearch(problem,limit)

```
{ Leaves = A Stack with MakeNode(initialState(problem)) as an element
  result = failure
  While not isEmpty (leaves)  do:
    { node = Pop(leaves)
      if GoalTest(problem,node) = true
        return Solution(node)
      if Depth[node] = limit
        result= cutoff
      else if node is not in ClosedList
        for each successor in Expand(node, problem)
          Push(successor, leaves)
    }
  return result
}
```

תרגיל הרצת DLS limit =2



סיכום הרצת Depth limited $l=2$



Depth-Limited Search

• **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?

• **אופטימאליות** – האם מוצא פתרון אופטימאלי ביחס לעלות המסלול?

• **סיבוכיות זמן** – כמה זמן צורך האג'?

• **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון.



Iterative Deepening Search

Iterative Deepening Search

- זוהי אסטרטגיה כללית. מפעילים אלג' אחר, לרוב DFS, בכל שלב עם מגבלת עומק גמישה יותר.

- בכל שלב מתחילים מהתחלה של החיפוש – כי בDFS בסיום חיפוש כל הקודקודים אינם שמורים.

Iterative Deepening Search

בכמה נעלה את הlimit ?



Iterative Deepening Search

```
IterativeDeepeningSearch(problem)
{
  for depth = 0 to  $\infty$ 
  {
    result = DepthLimitedSearch (problem, depth)
    If result  $\neq$  cutoff
      return result
  }
}
```

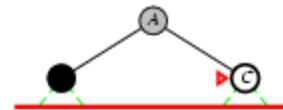
Iterative Deepening search $l = 0$

Limit = 0



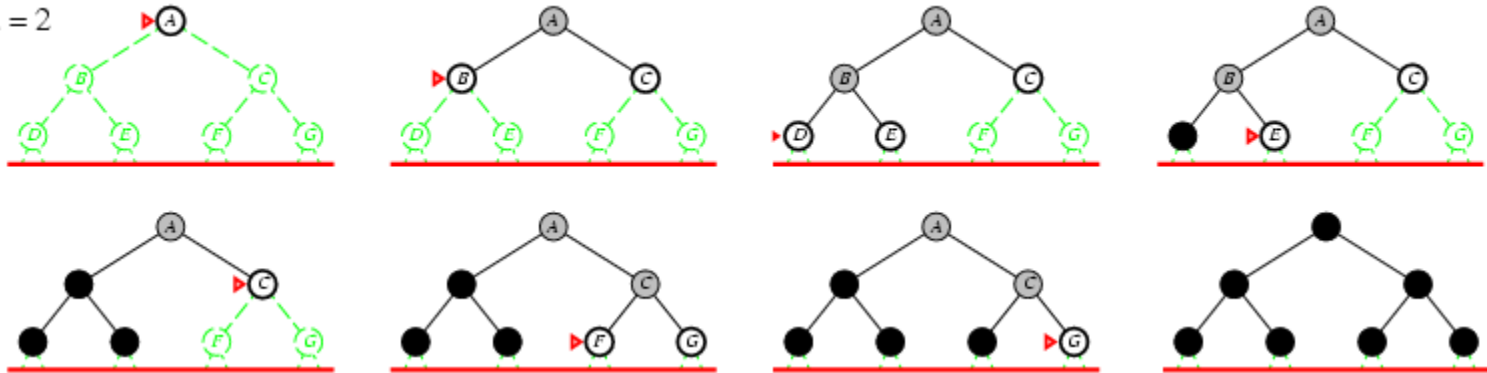
Iterative Deepening search $l = 1$

Limit = 1



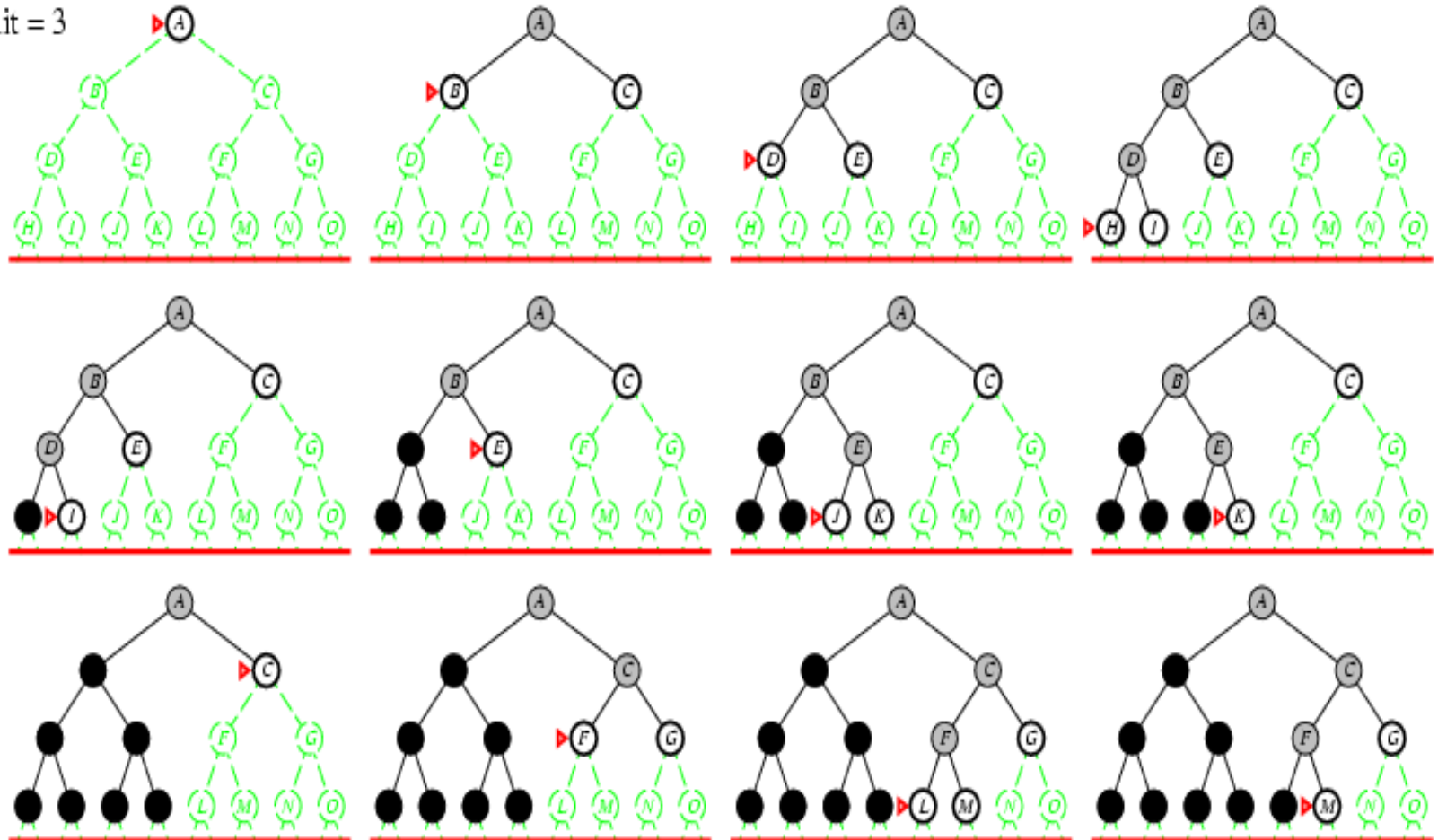
Iterative Deepening search $l = 2$

Limit = 2

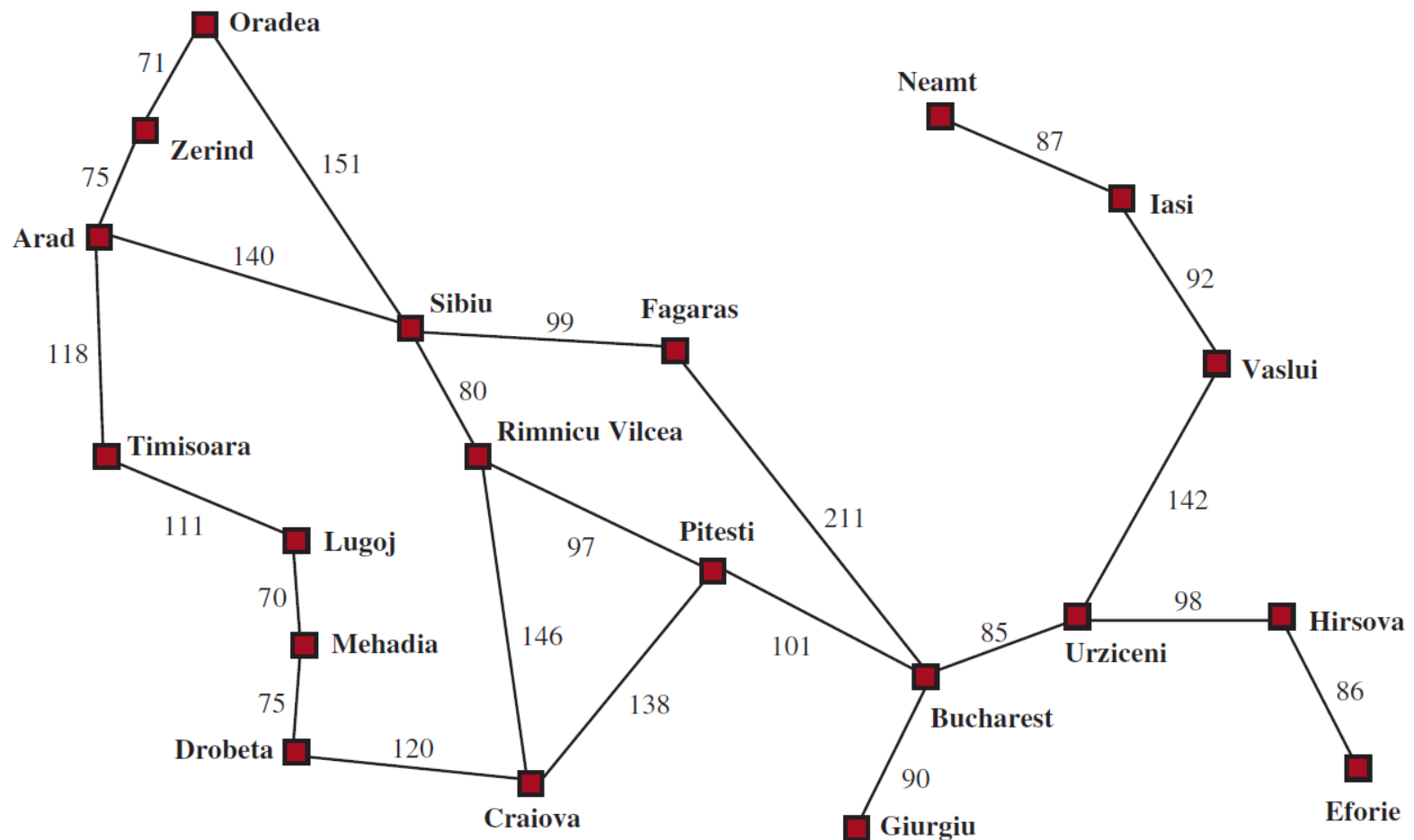


Iterative Deepening search $l = 3$

Limit = 3



תרגיל הרצת Iterative Deepening



Iterative Deepening search

- # nodes generated in a search to depth d , branching factor b :
- $N_{BFS} = b^0 + b^1 + b^2 + \dots + b^d$
- $N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 2b^{d-1} + b^d$

For $b = 10$, $d = 5$,

- $N_{BFS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$
- $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$
- Overhead = $(123,456 - 111,111) / 111,111 = 11\%$

Iterative Deepening Search -example

Iterative Deepening

limit = 0



limit = 1

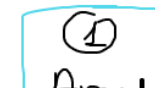


limit = 2

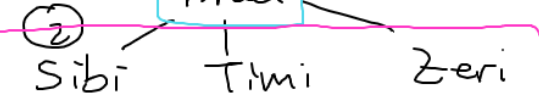


limit 0 1 2 3

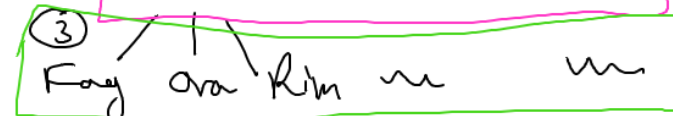
limit 0



limit 1



limit 2



limit 3



d = 3

$$1 \cdot 4 + b \cdot 3 + b^2 \cdot 2 + b^3 \cdot 1$$

$$1 \cdot d + b \cdot d + b^2(d-1) + \dots + b^{d-1} \cdot 1$$

Iterative Deepening search

- נראה בזבזני שבכל שלב כל הקודקודים מפותחים מחדש, אך כיוון שבכל שלב, העלים הם החלק המשמעותי בעץ, פיתוח חוזר של קודקודים פנימיים לא מעלה סיבוכיות הזמן. יש בזה חסכון במקום.

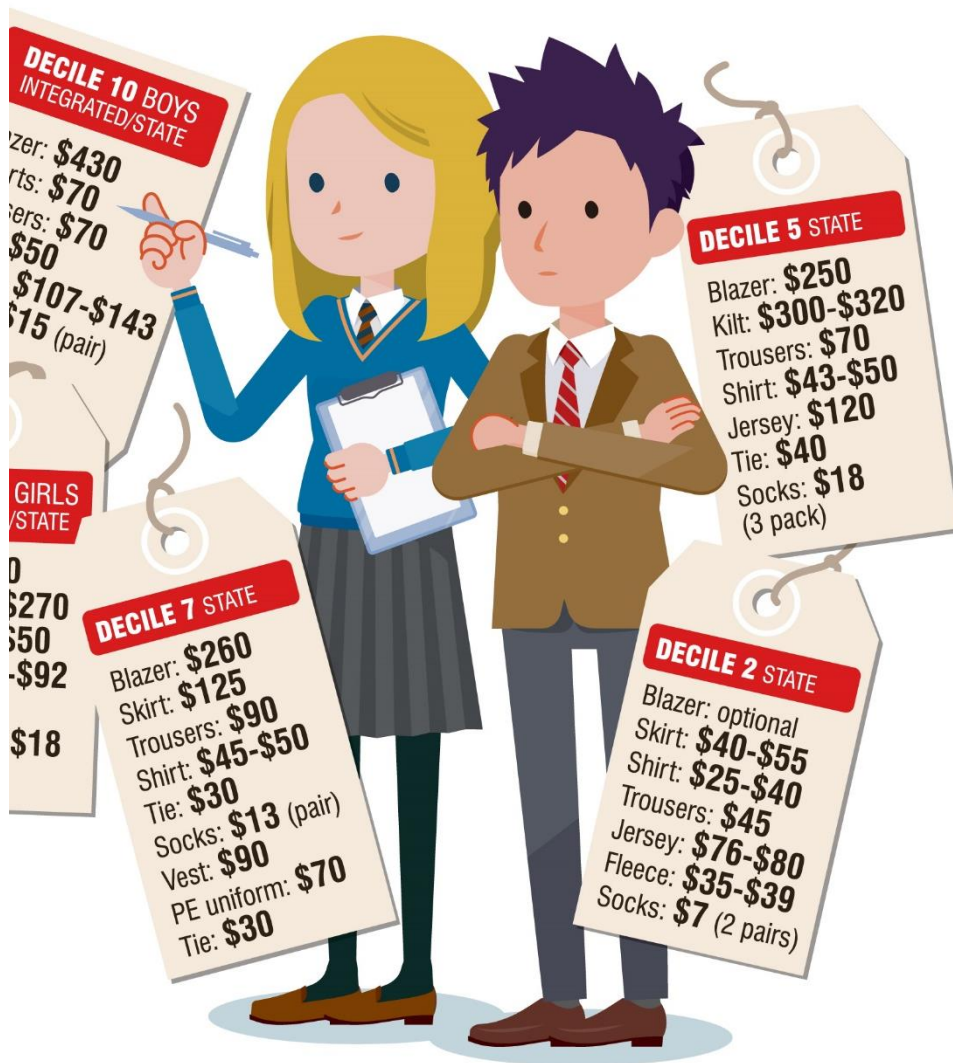
Iterative Deepening Search

• **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?

• **אופטימאליות** – האם מוצא פתרון אופטימאלי
ביחס לעלות המסלול?

• **סיבוכיות זמן** – כמה זמן צורך האג'?

• **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון



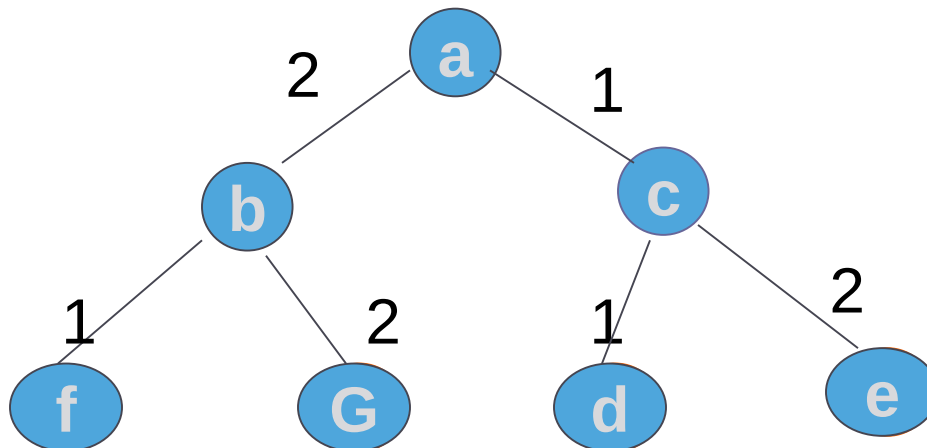
UNIFORM-COST SEARCH

Uniform-Cost Search

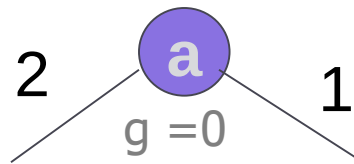
- מפתח קוד' בעל עלות מסלול נמוכה ביותר ראשון.
- שומר העלים בתור שמסודר לפי עלות מסלול.
- מה קורה כשעלות כל שלב זהה?

דוגמאות של-Uniform Cost Search

נניח עץ דוגמא בעל מחירי קשתות שונים, המיוצגים ע"י המספרים ליד הקשתות
מחפשים מסלול קצר ביותר מ a ל G.

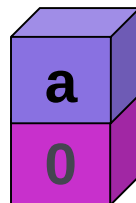


דוגמא ל - Uniform Cost Search

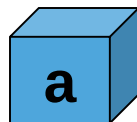
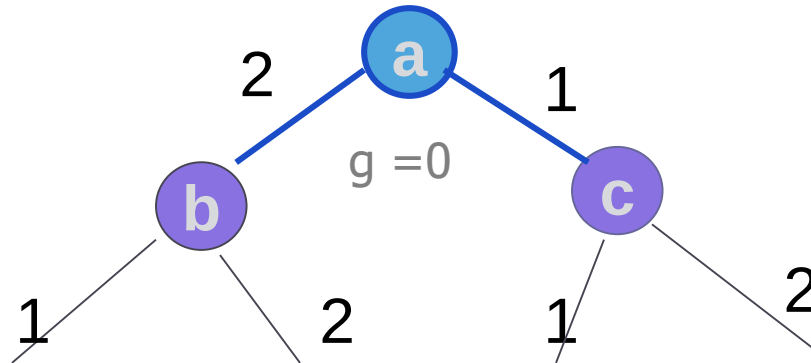


Closed list:

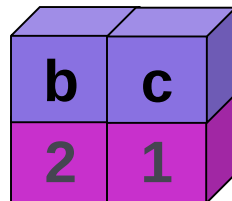
Open list:



דוגמה ל-Uniform Cost Search

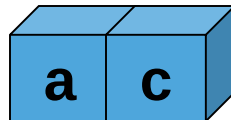
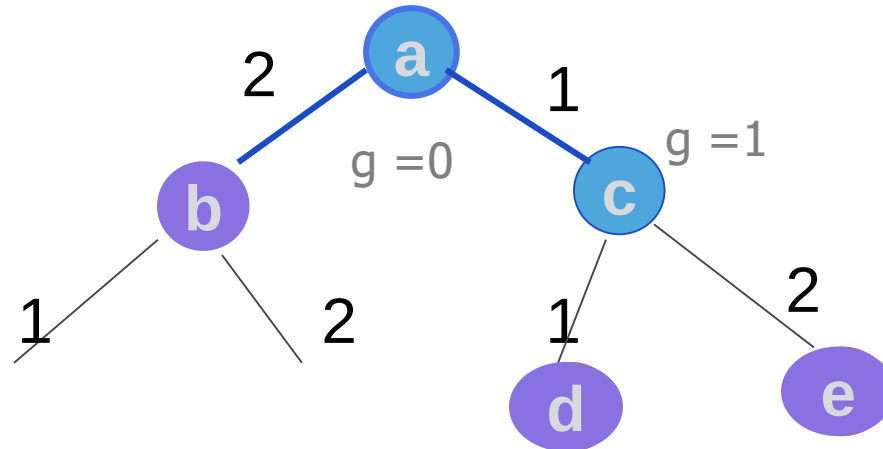


Closed list:

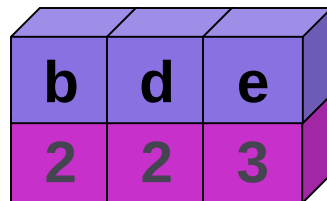


Open list:

דוגמא ל-Uniform Cost Search

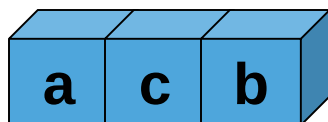
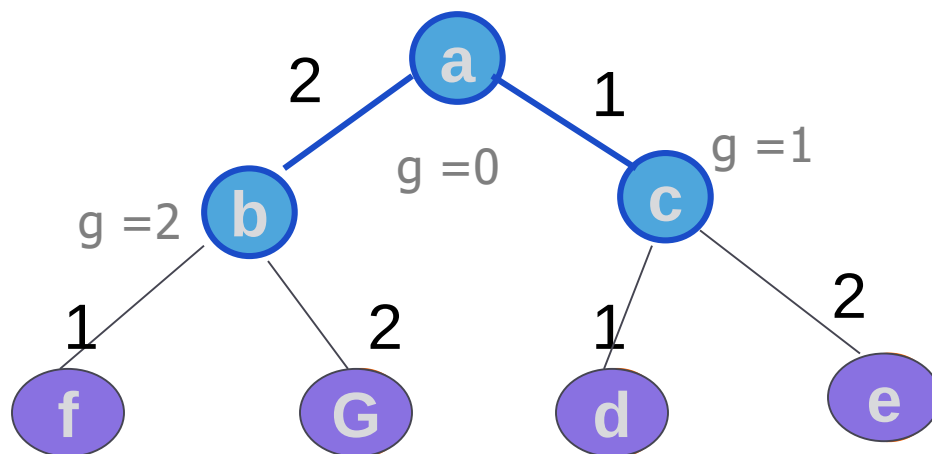


Closed list:

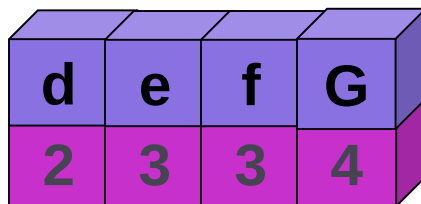


Open list:

דוגמא ל-Uniform Cost Search

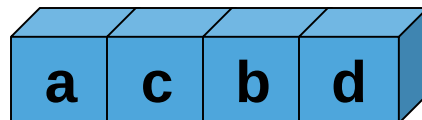
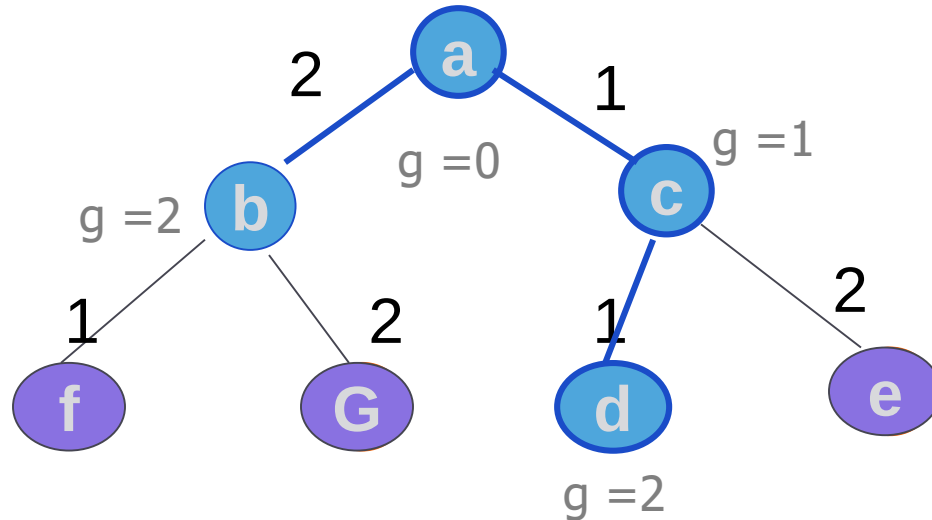


Closed list:

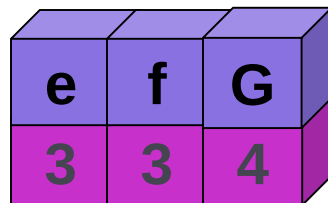


Open list:

דוגמא ל- Uniform Cost Search

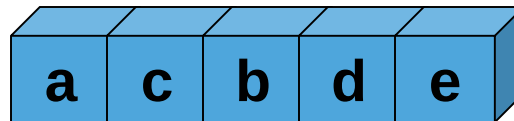
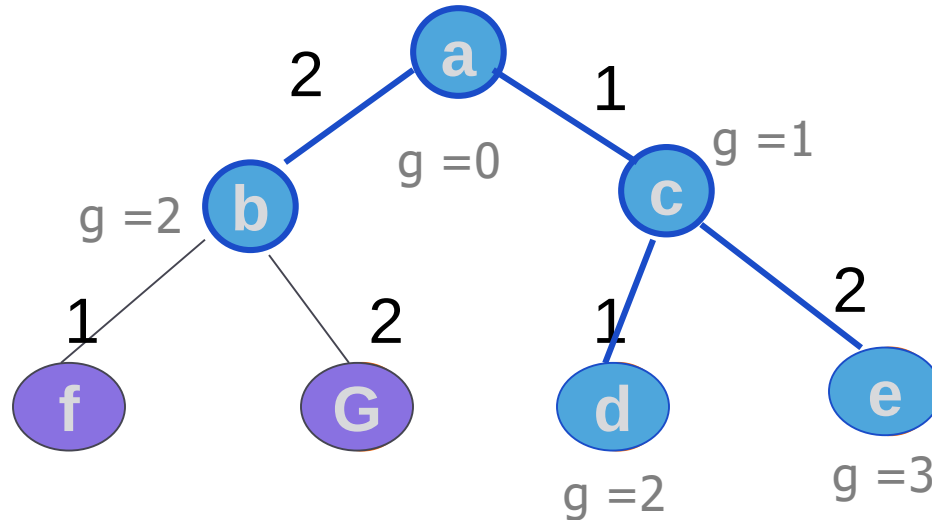


Closed list:

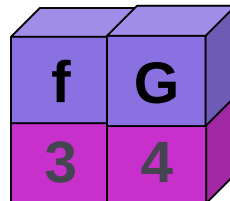


Open list:

דוגמא ל- Uniform Cost Search

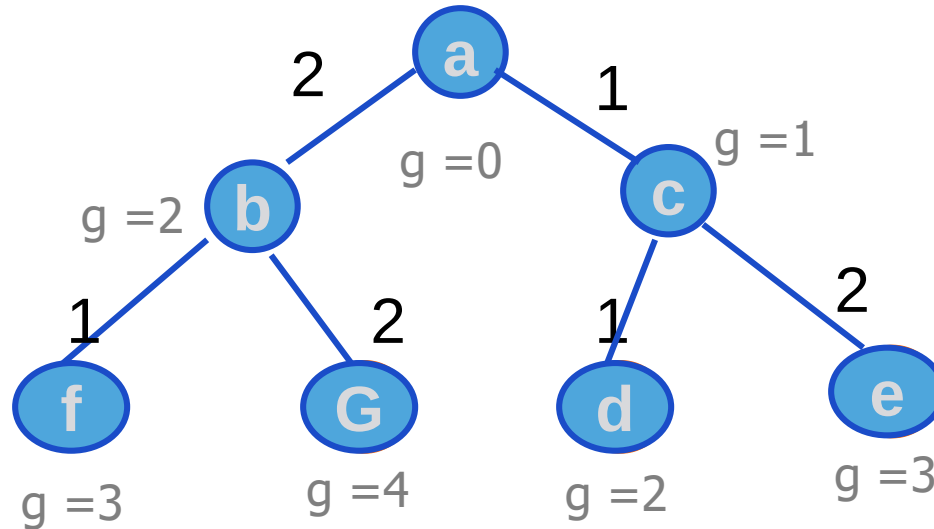


Closed list:



Open list:

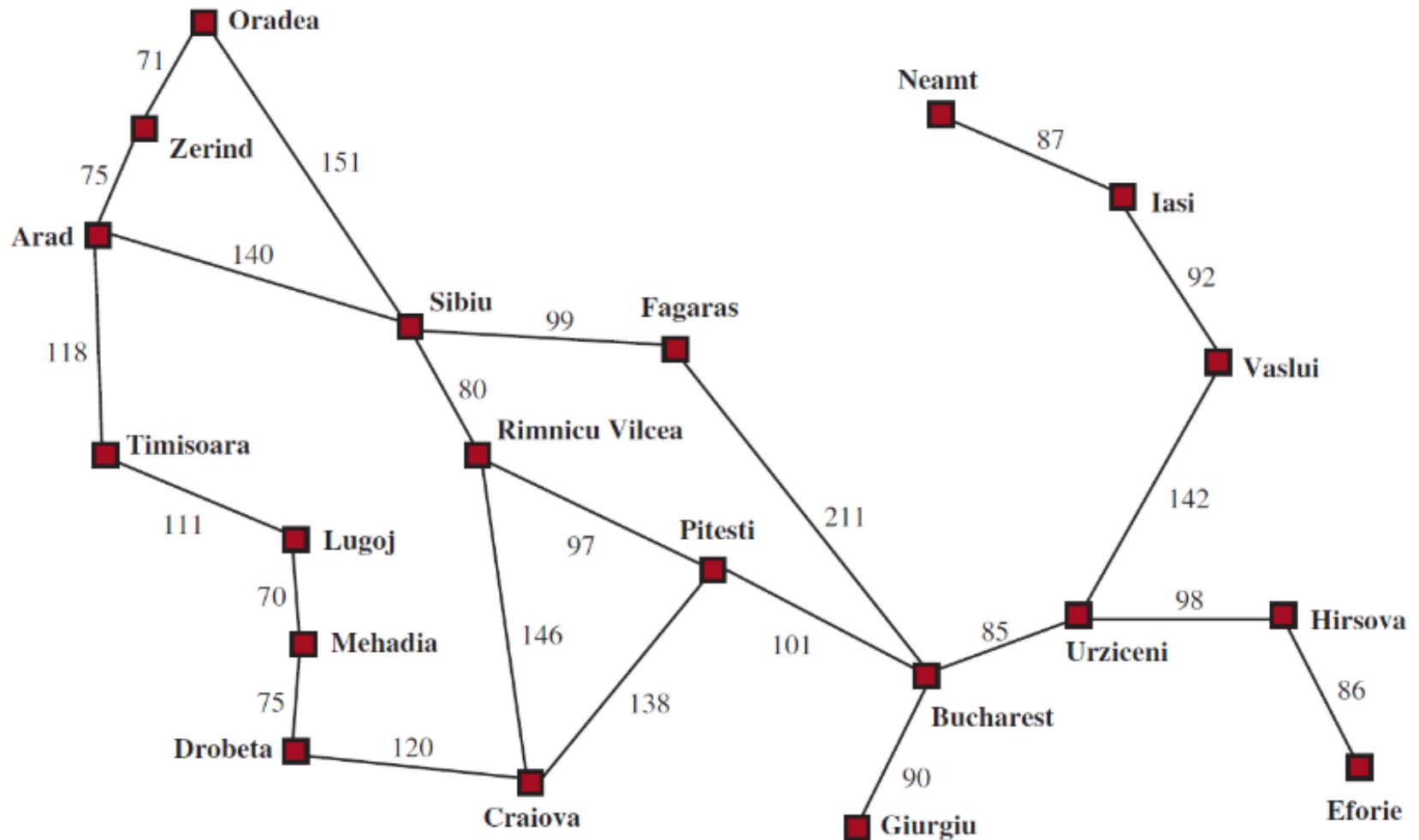
דוגמא ל - Uniform Cost Search



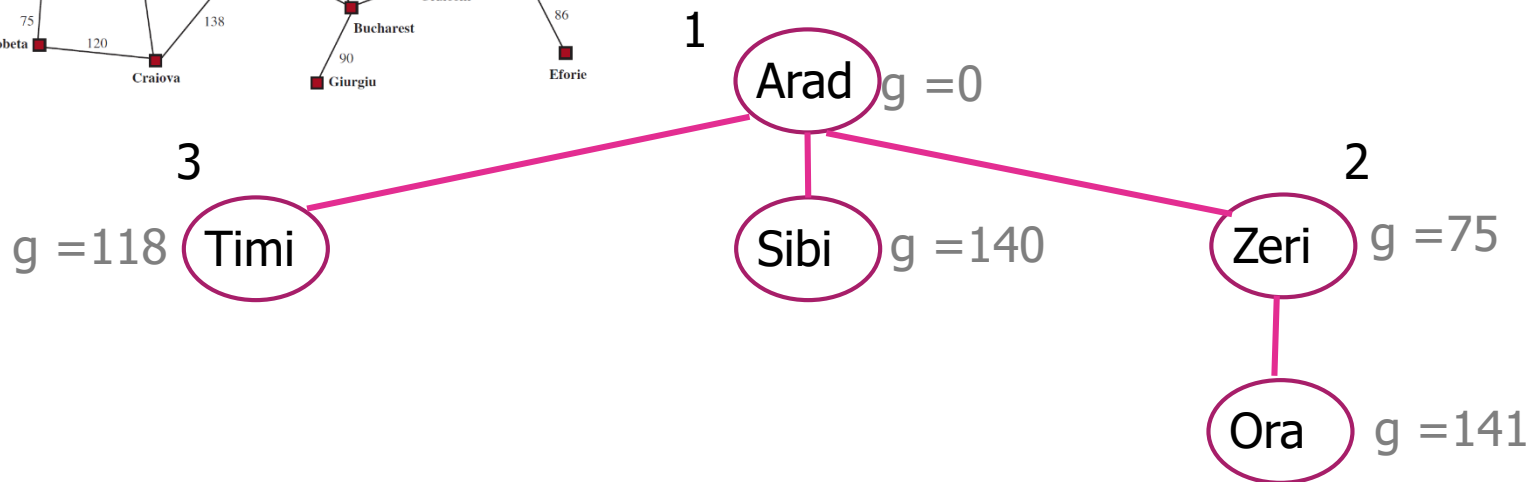
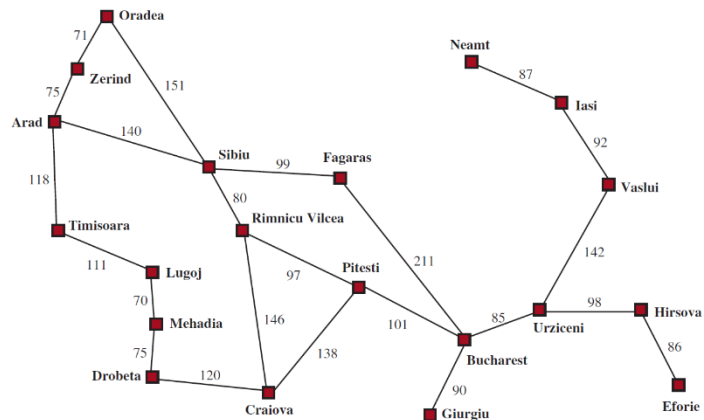
Closed list:

Open list:

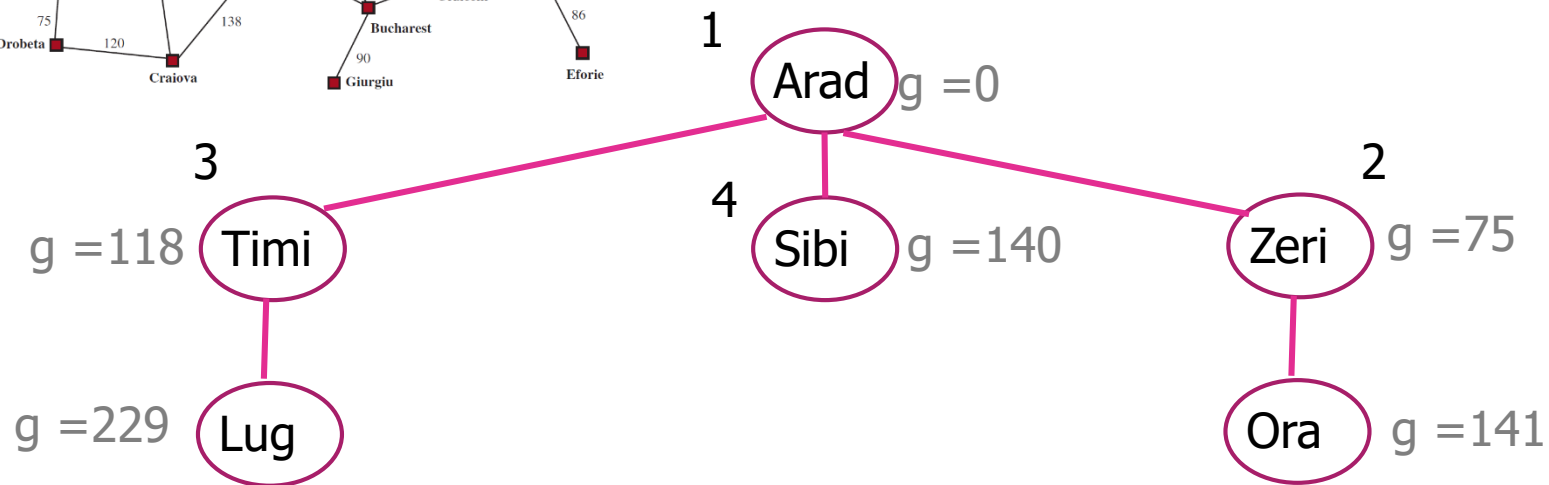
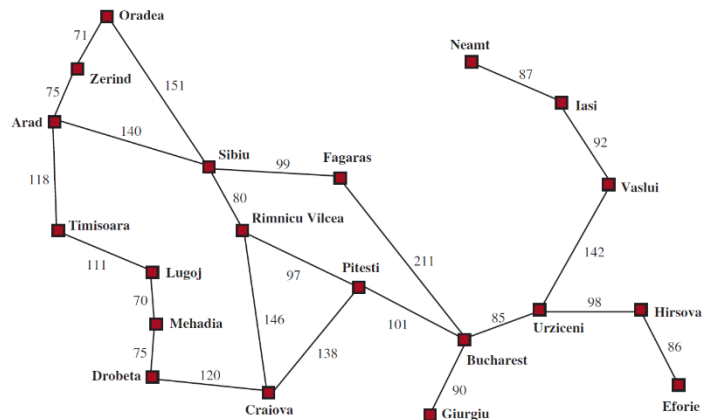
תרגיל הרצת UCS



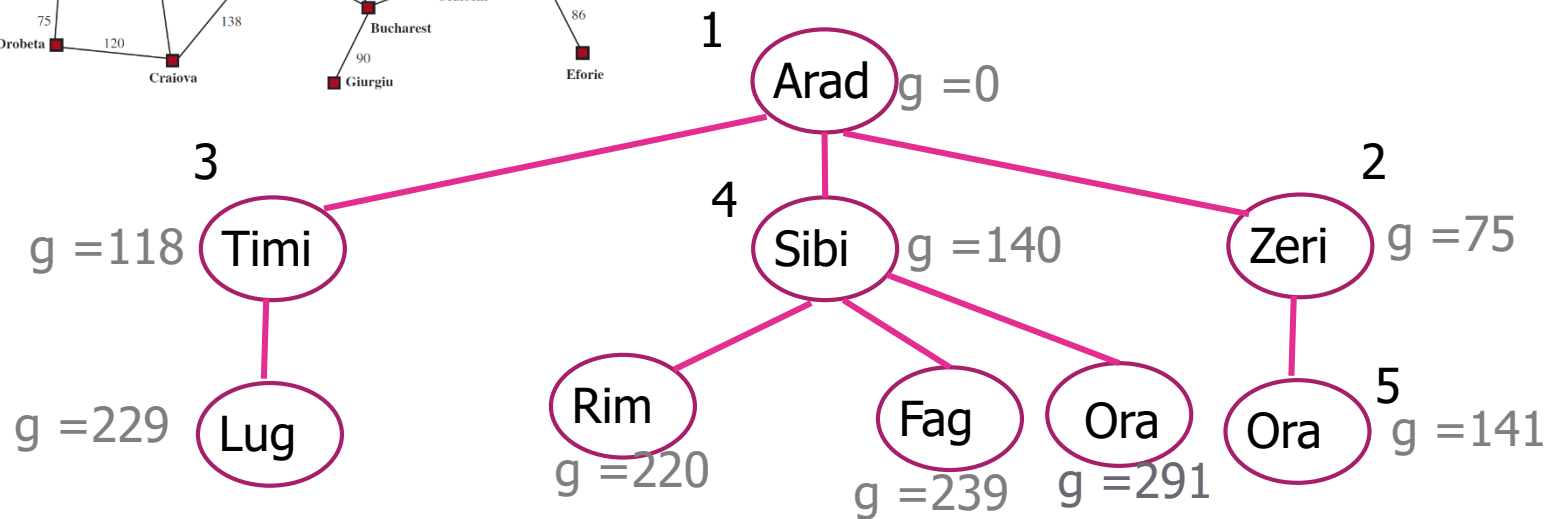
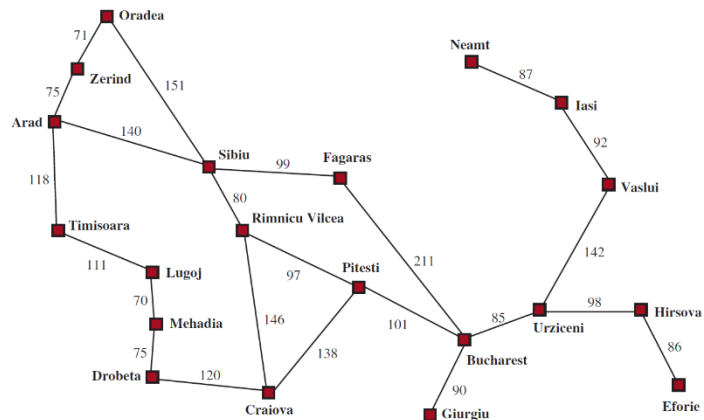
הרצת UCS



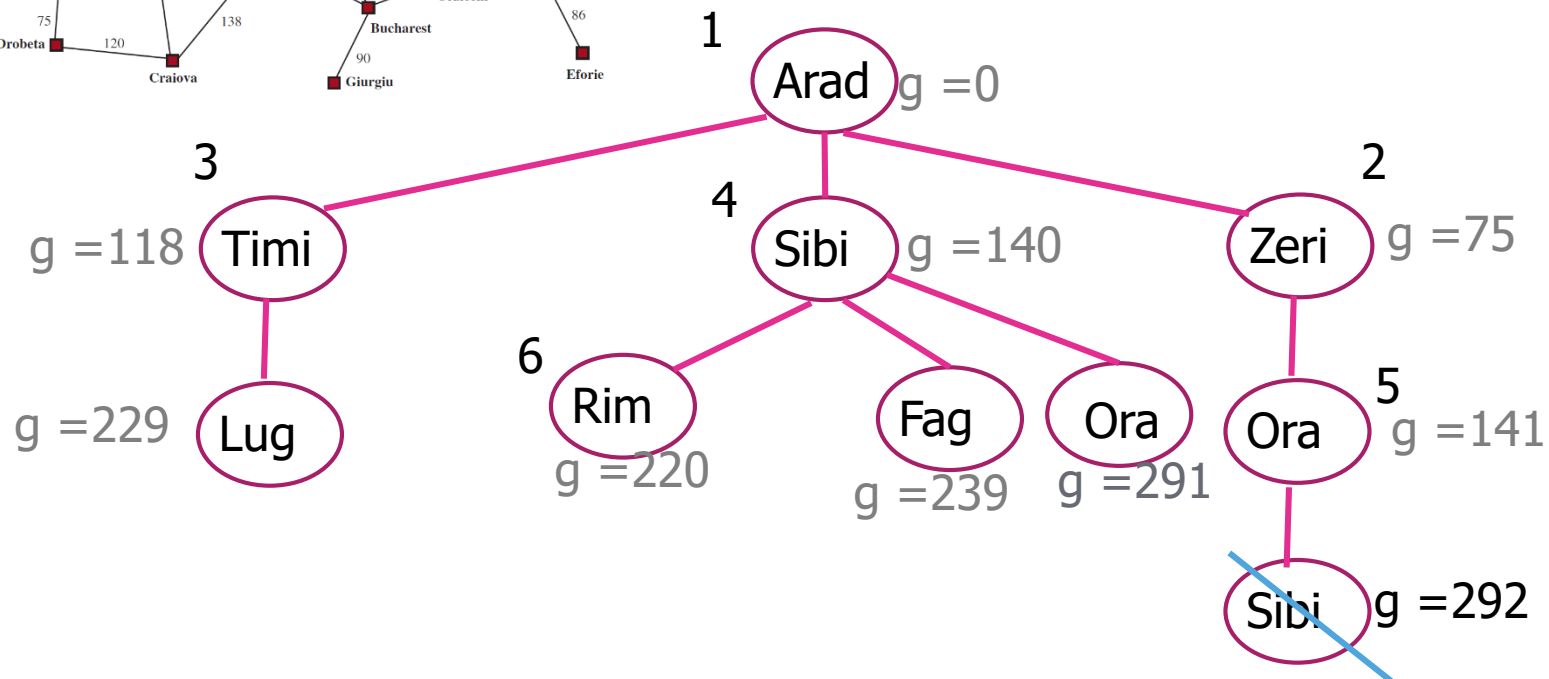
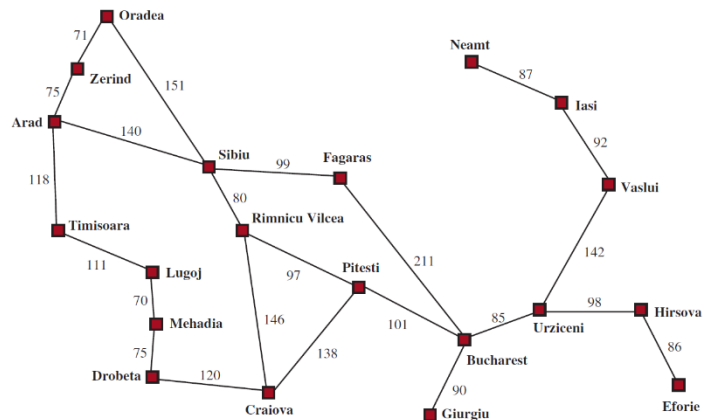
הרצת UCS



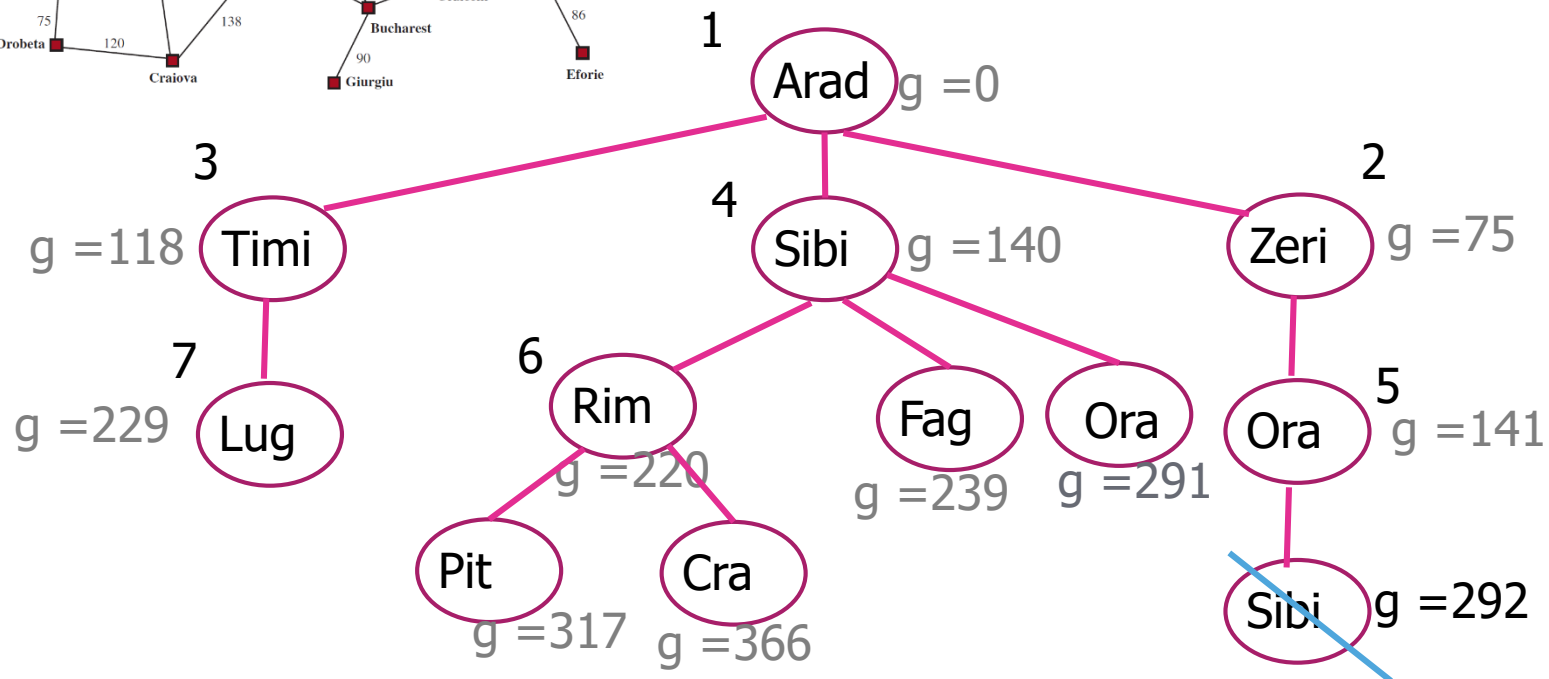
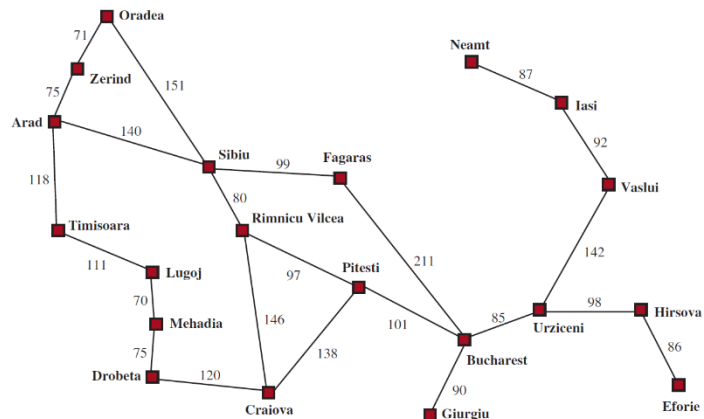
הרצת UCS



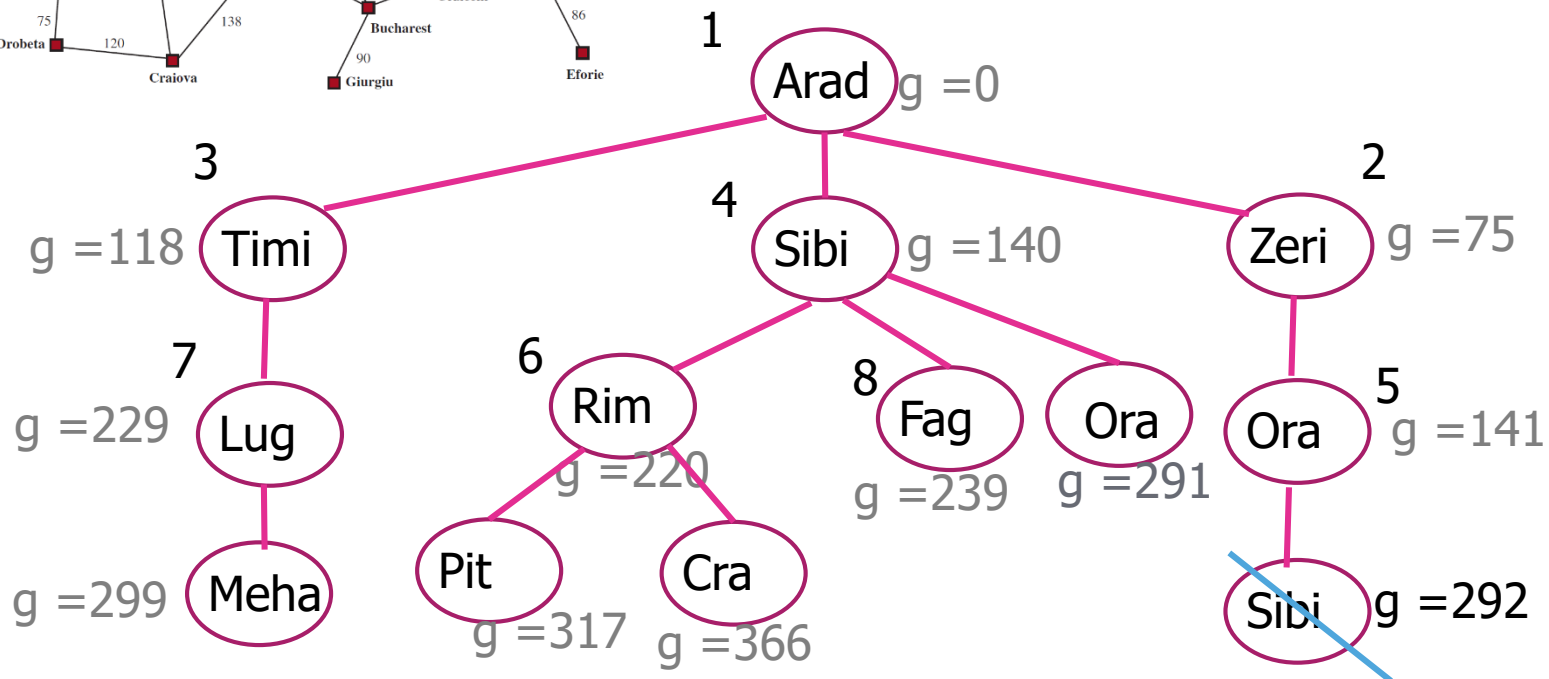
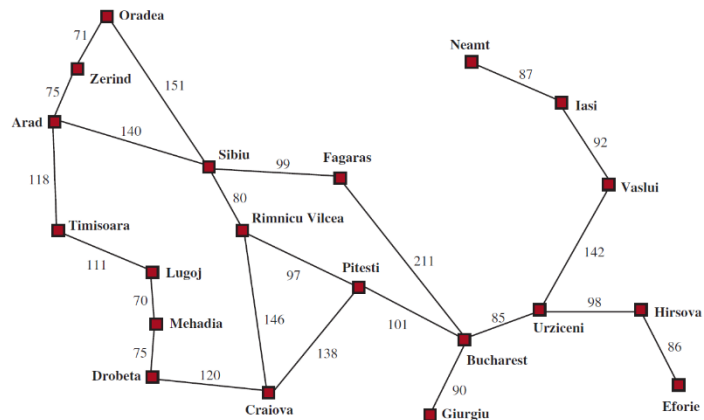
הרצת UCS



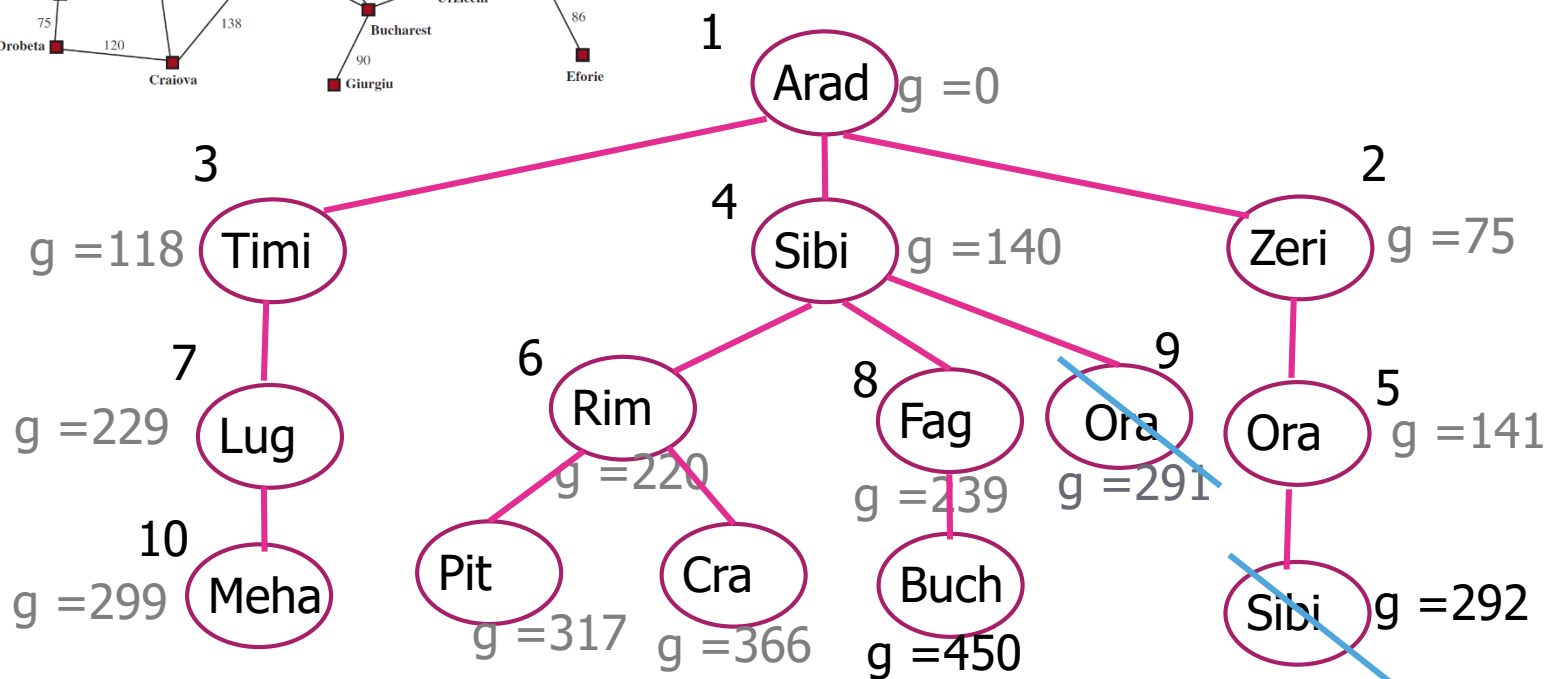
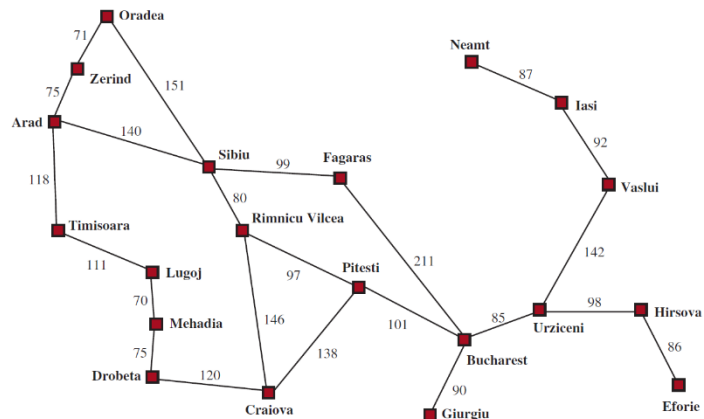
הרצת UCS



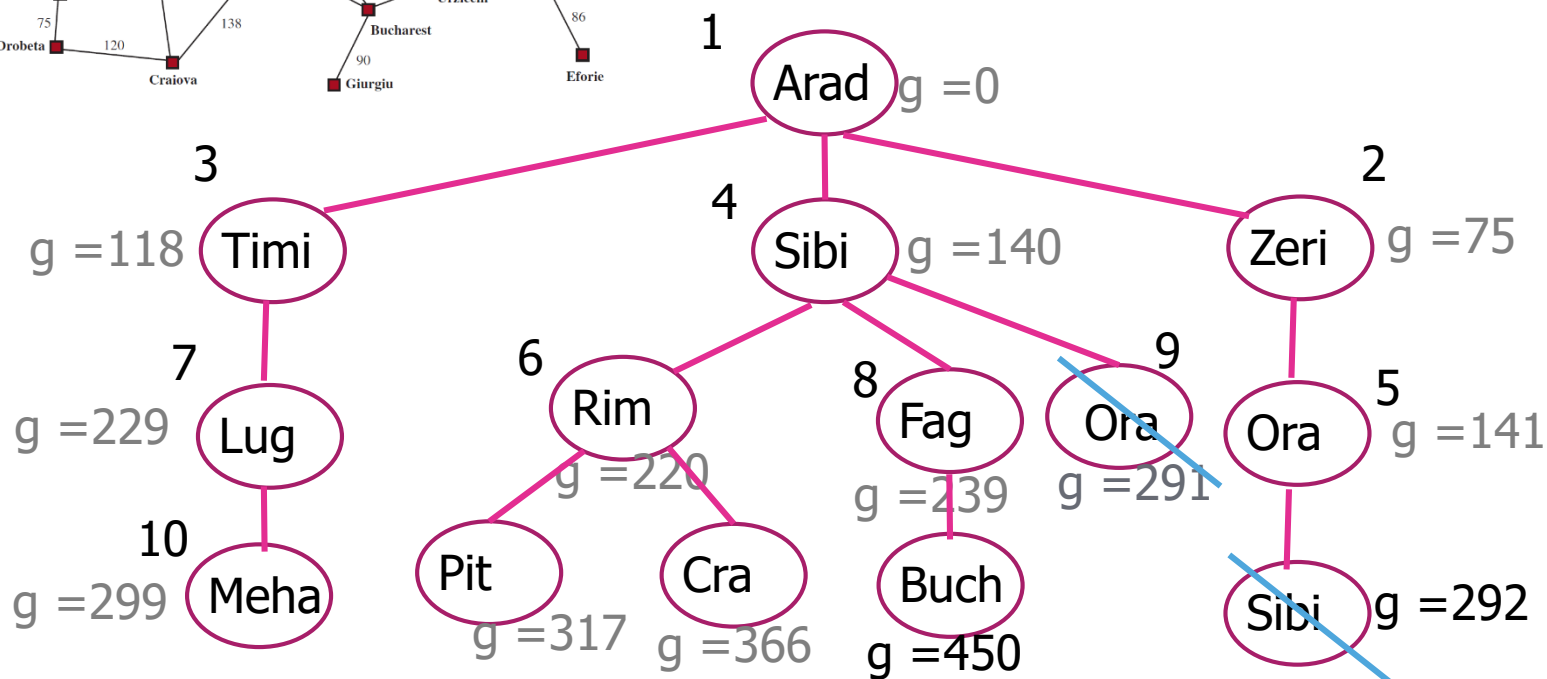
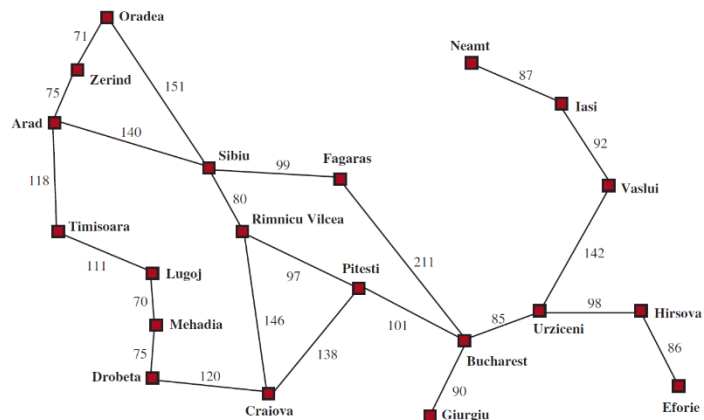
הרצת UCS



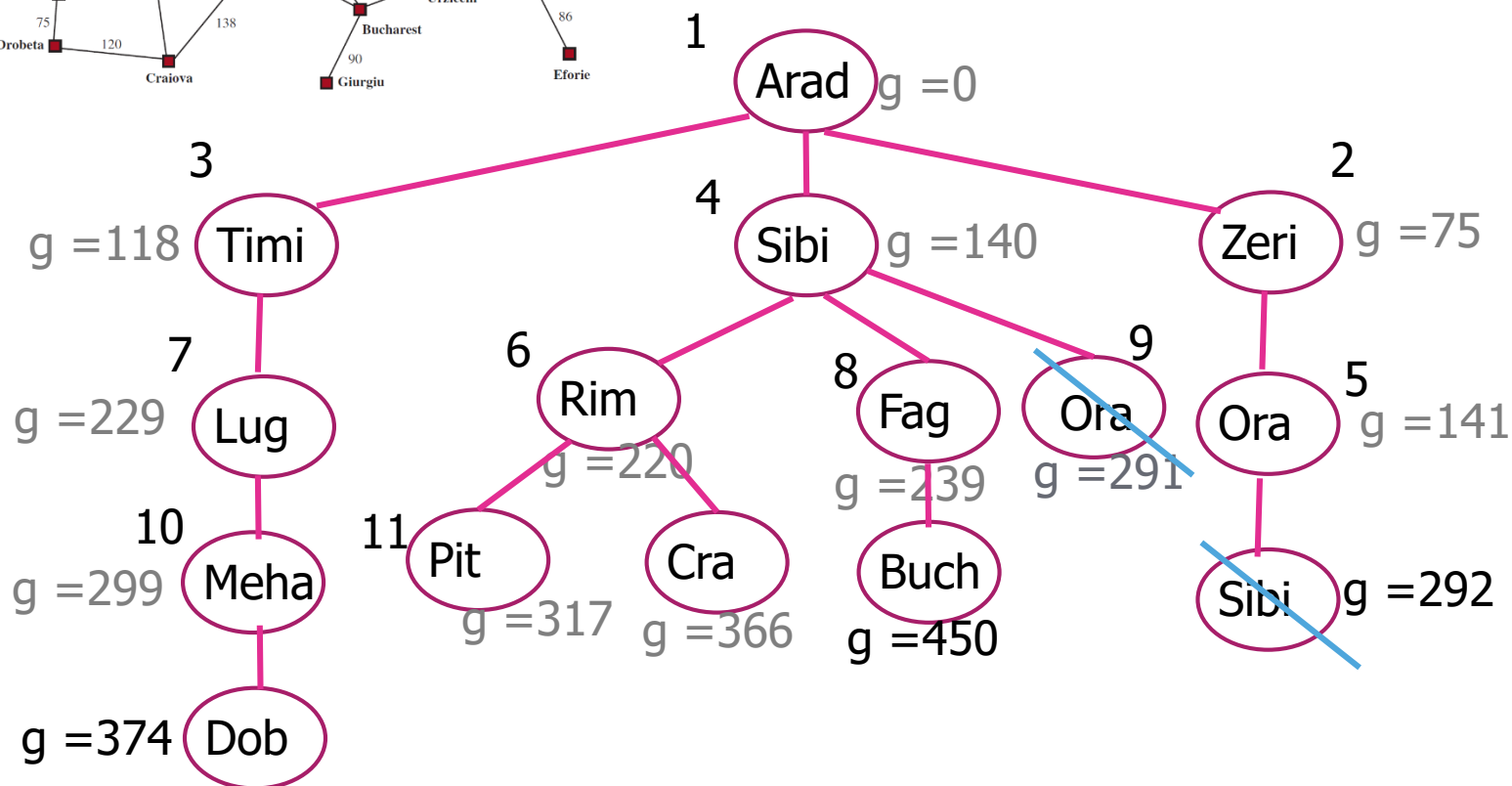
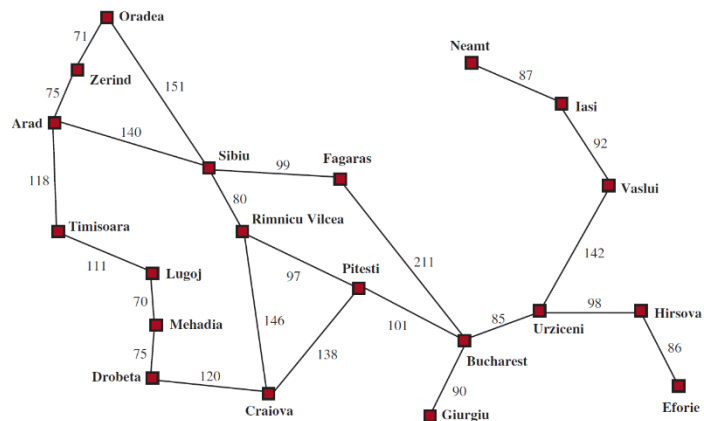
הרצת UCS



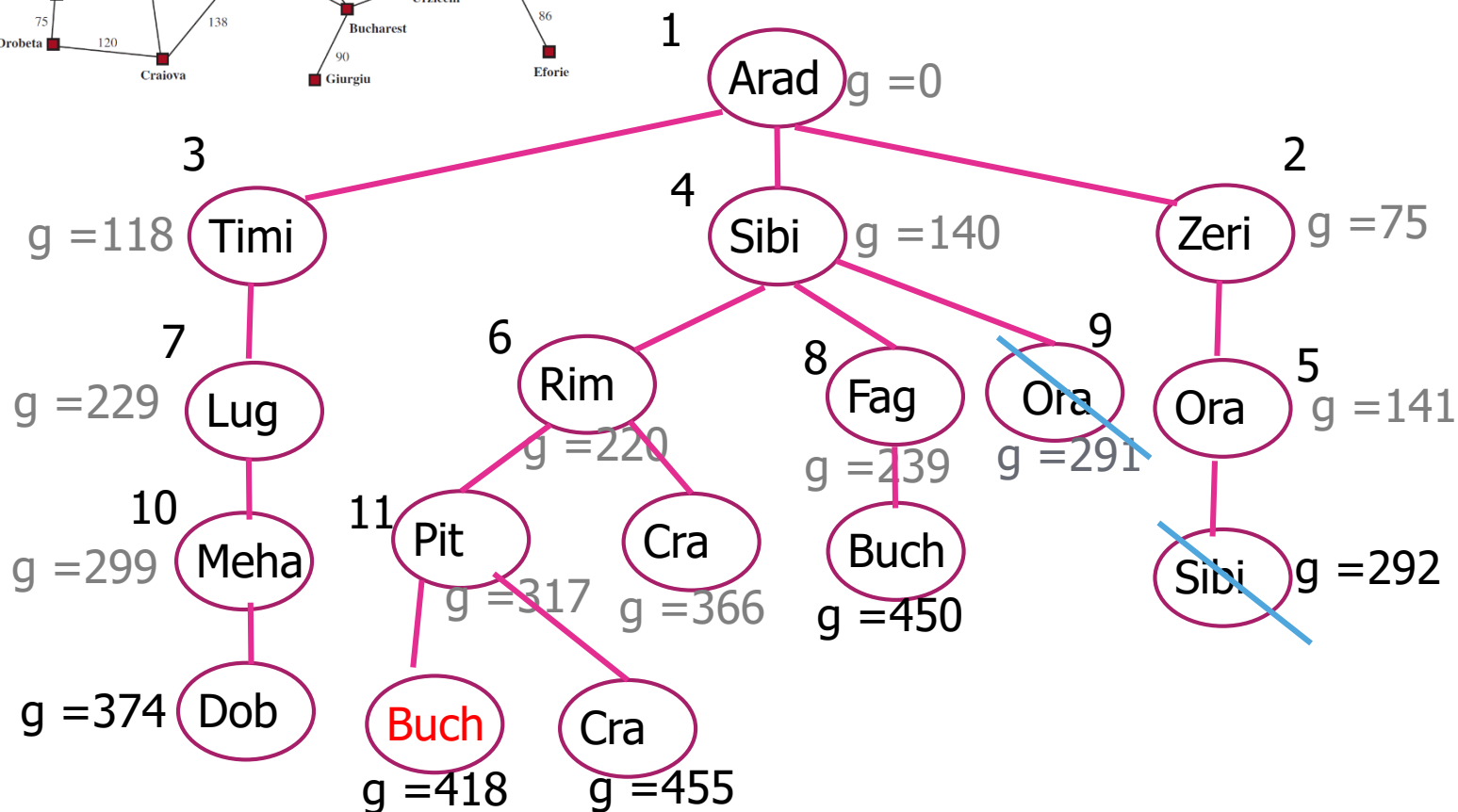
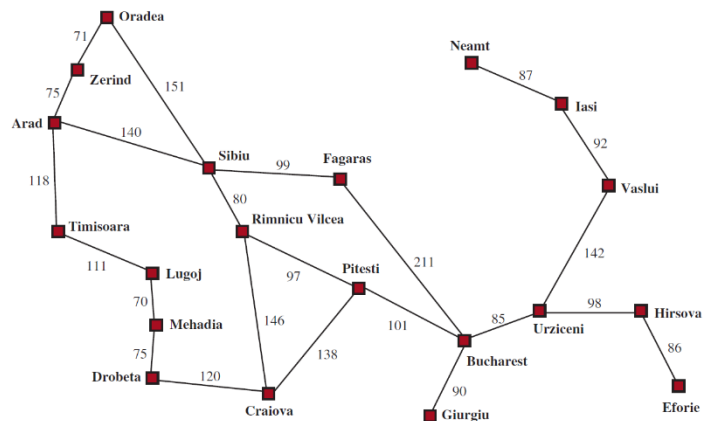
הרצת UCS



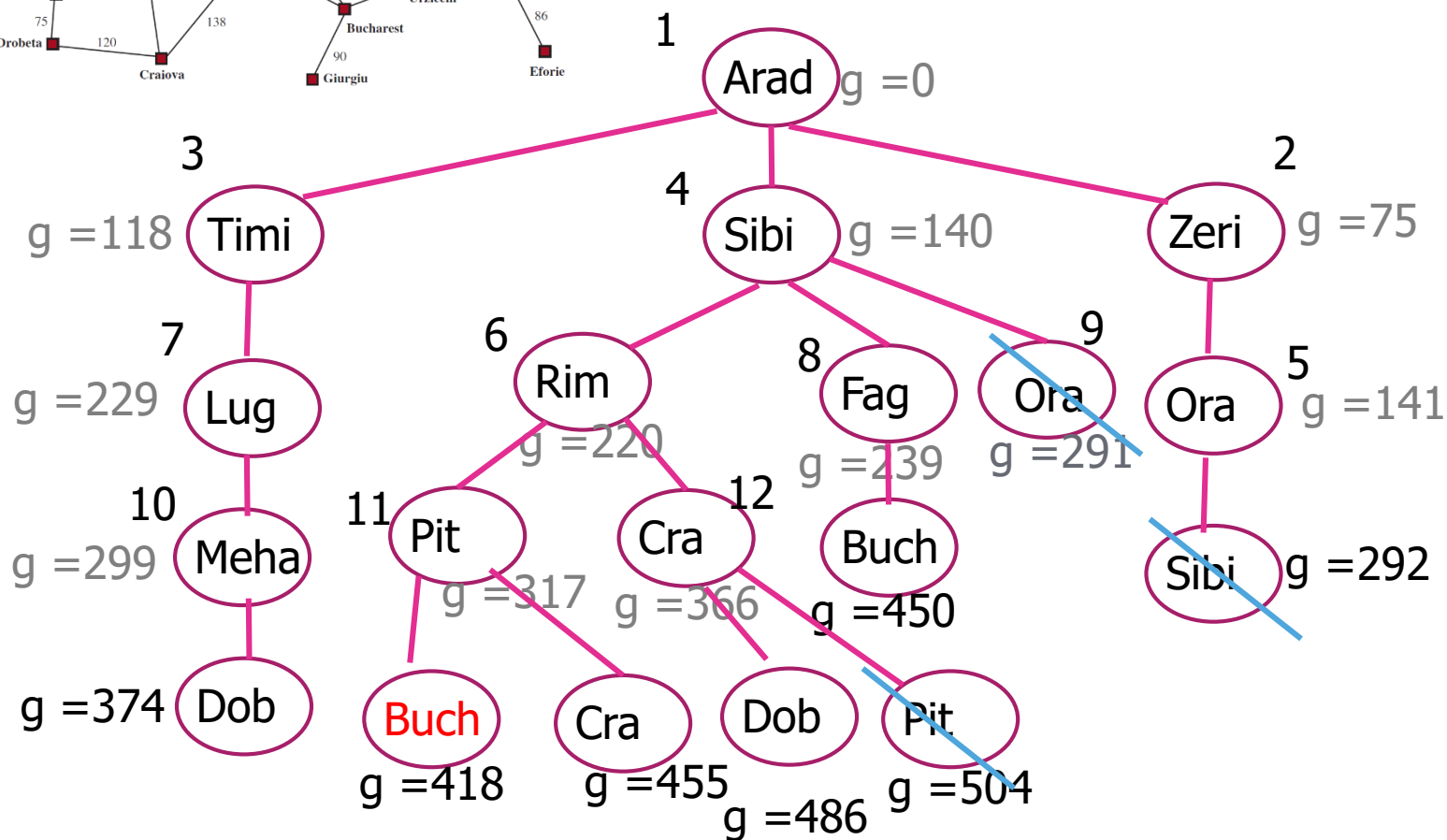
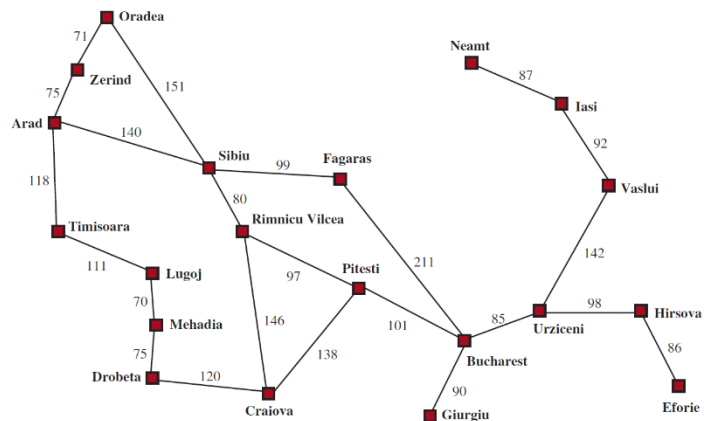
הרצת UCS



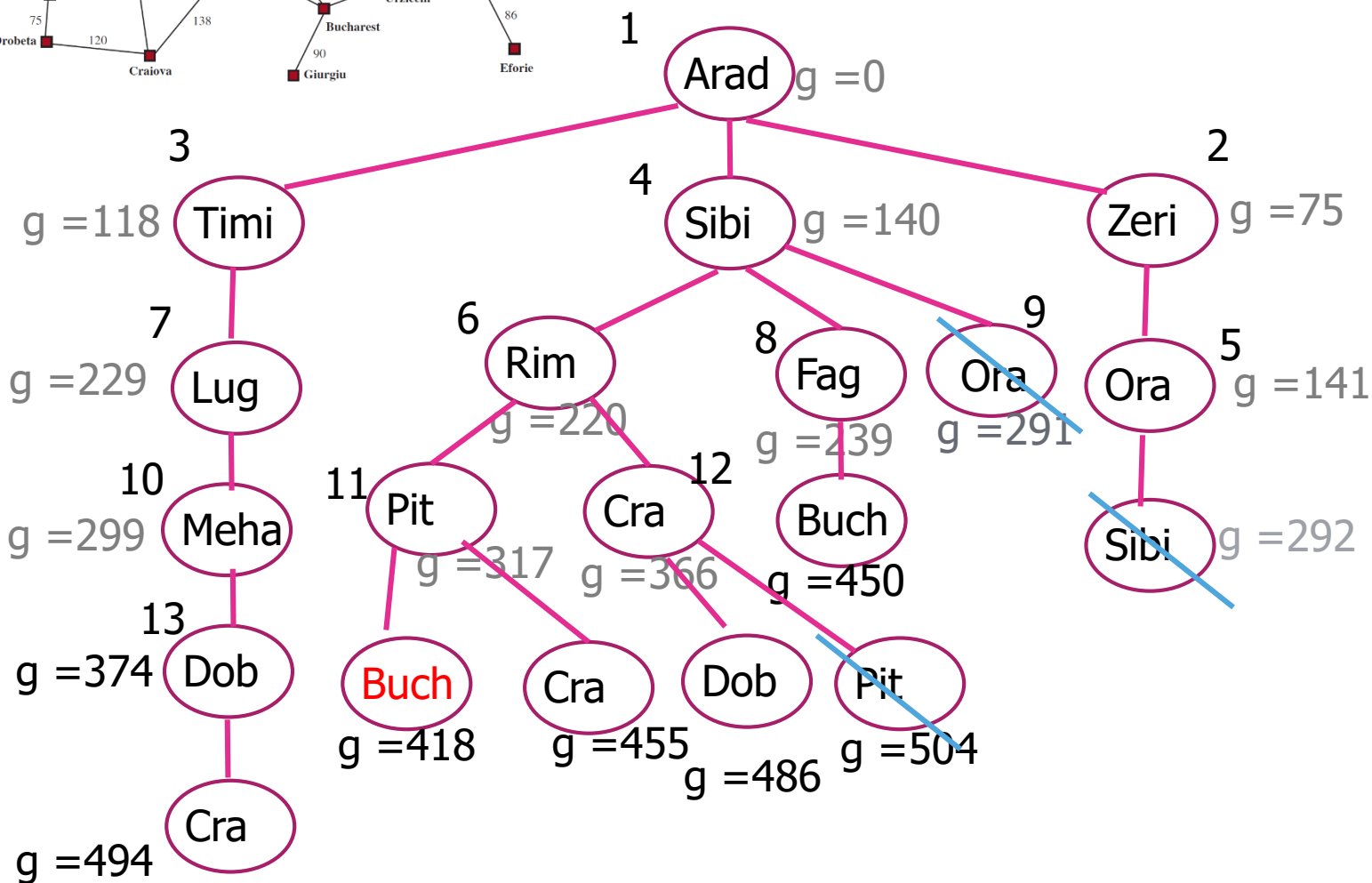
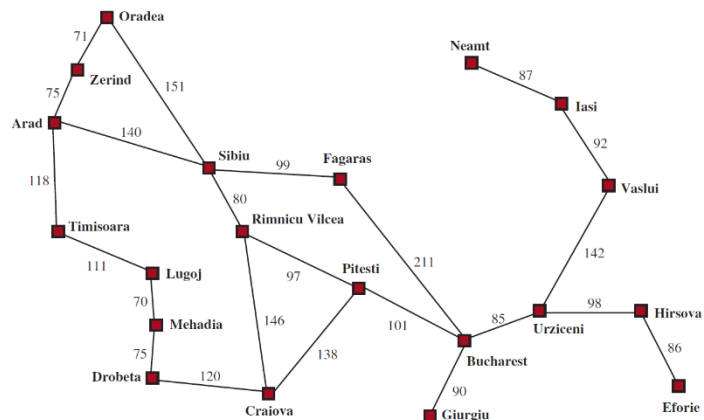
הרצת UCS



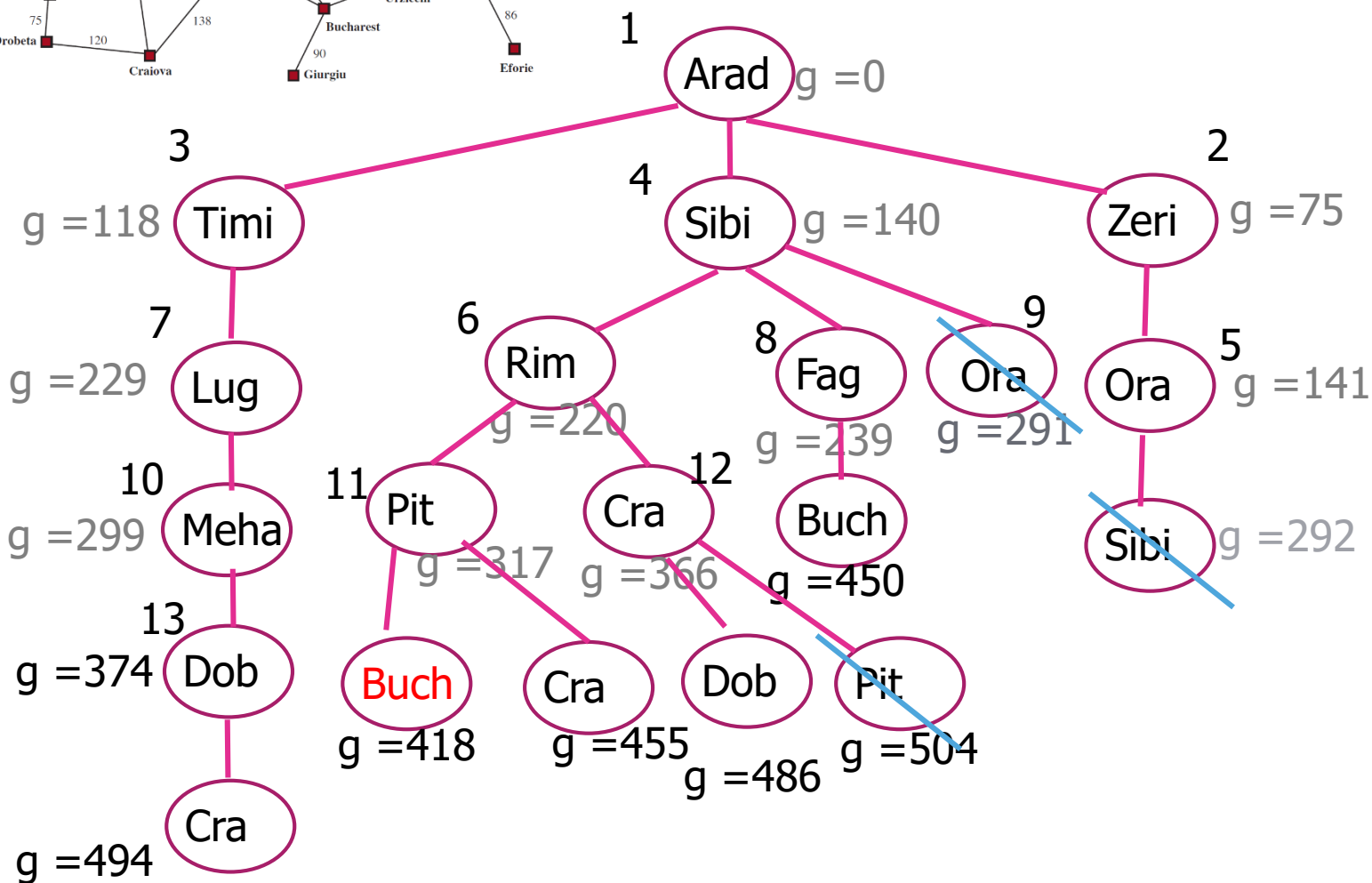
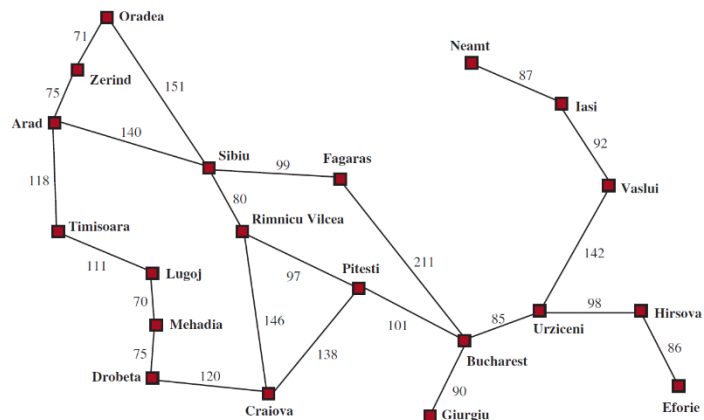
הרצת UCS



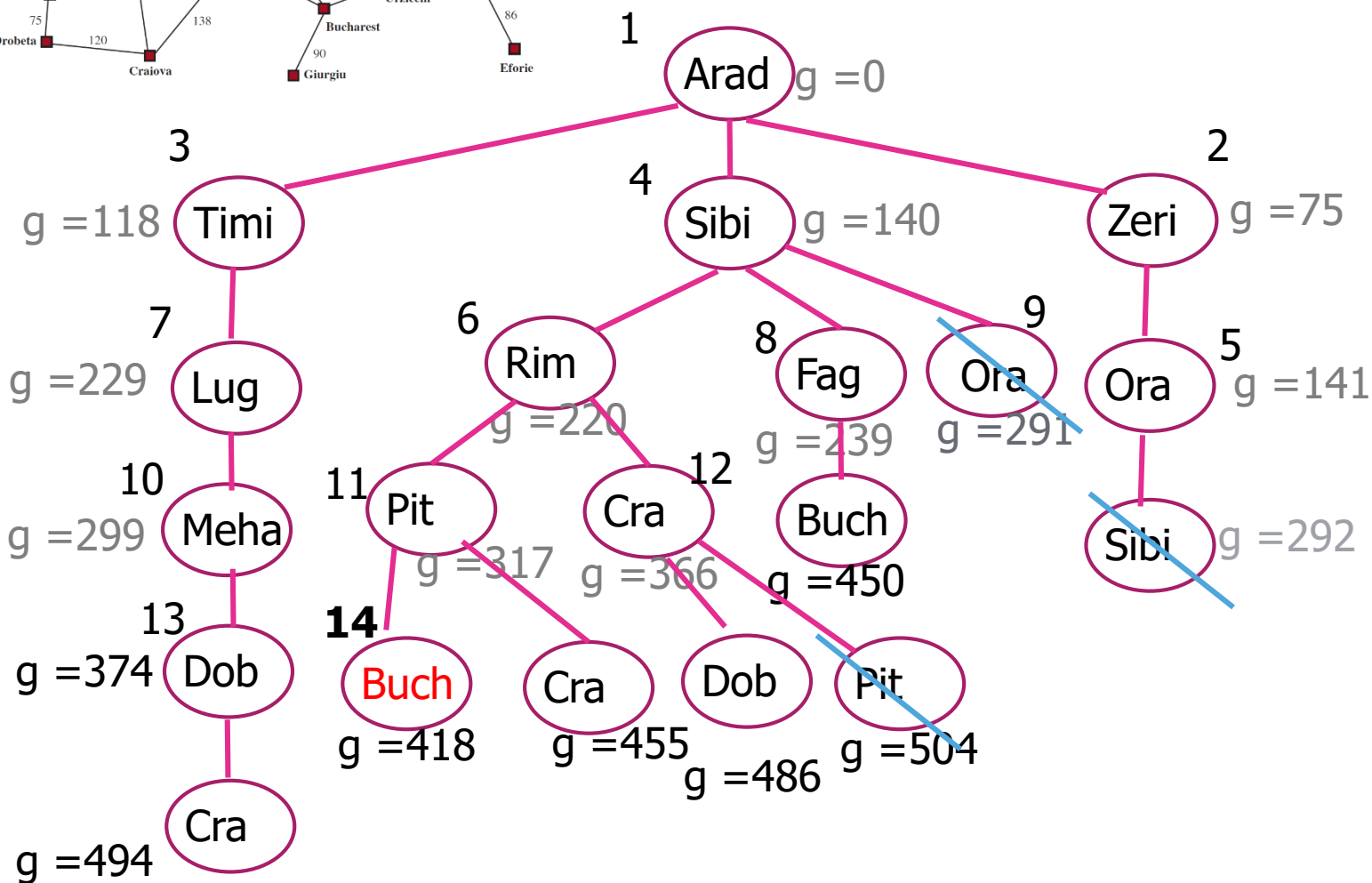
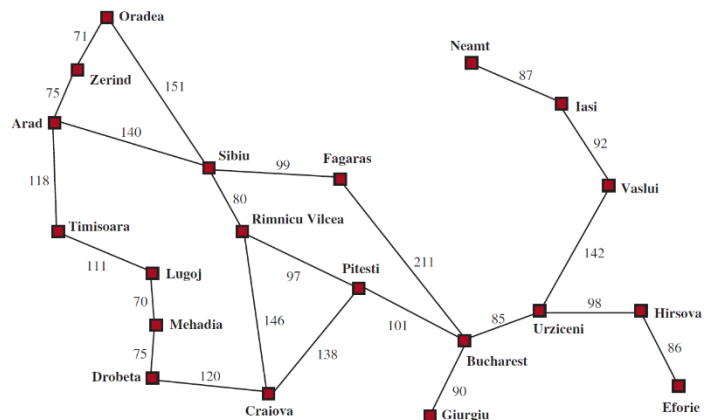
הרצת UCS



הרצת UCS



הרצת UCS



Uniform-Cost Search

• **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?

• **אופטימאליות** – האם מוצא פתרון אופטימאלי ביחס לעלות המסלול?

• **סיבוכיות זמן** – כמה זמן צורך האג'?

• **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון